

Systems Thinking Project

Acrobot: Modelling, Stability, Control and Swing-Up

Full Project Report

Author: Bhargav
ID: 2025102061

Author: Sreekar
ID: 2025102074

Author: Kaushik
ID: 2025102071

Author: Divijat
ID: 2025102040

November 11, 2026

Contents

1 Introduction **4**

1.1 What is an Acrobot? 4

1.2 Objective 4

2 System Modelling **4**

2.1 Model 4

2.2 Equations of Motion 5

2.3 Derivation of the Model 5

2.4 State-Space Form 6

3 Equilibria **6**

3.1 Conditions 6

3.2 Solutions 6

4 Linearization and Transfer Functions **7**

4.1 Deviation Variables and Jacobians 7

4.2 Linear State-Space Model 7

4.3 Transfer Functions 7

5 Simulation Environment and Model Verification **8**

5.1 PyBullet Model 8

5.2 Verification 8

5.3 Interactive Simulator 9

6 Stability Analysis **10**

6.1 Stability Criteria 10

6.2 Poles of the Transfer Functions = Eigenvalues of A 10

6.3 Characteristic Polynomial 10

6.4 Link to the Potential Energy 10

6.5 Stability of Each Equilibrium 11

6.6 Simulation Evidence 11

7 Feedback from a Single Measured Angle **13**

7.1 Static or PD Feedback 13

7.2 Dynamic (Observer-Based) Feedback 13

8 Controller Design Around the Equilibria **14**

8.1 Controller Design 14

8.2 Linear Quadratic Regulator (LQR) 15

8.2.1 Choice of R . . . 15
8.3 Pole Placement at E_3 (Comparison) . . . 15
8.4 E_1: PD on the Elbow Angle Only . . . 16
8.5 Designed Controllers . . . 16

9 Performance Limits under the Torque Limit **16**
9.1 Guaranteed Region . . . 16
9.2 Simulated Region of Attraction . . . 16
9.3 Time Responses . . . 17

10 Swing-Up Control **18**
10.1 Energy Target . . . 18
10.2 Swing-Up Controller . . . 18
10.3 Hybrid Switching . . . 19
10.4 Choice of Gains . . . 19
10.5 Results . . . 19
10.6 Why It Works, and Why Other Gains Fail . . . 20

11 Learning-Based Swing-Up **21**
11.1 Why Reinforcement Learning? . . . 21
11.2 Hybrid Structure . . . 21
11.3 Environment . . . 21
11.4 Reward Design . . . 22
11.5 Algorithm: Soft Actor-Critic (SAC) . . . 22
11.6 Training . . . 23
11.7 Results: Correct Model . . . 23
11.8 Results: Wrong Model . . . 24
11.9 Making the Policy Robust . . . 24
11.10Discussion . . . 26

12 Simulink Implementation and Cross-Validation **26**
12.1 Common Plant Structure . . . 27
12.2 Models . . . 27
12.3 Results . . . 28
12.4 Cross-Validation with the Python Model . . . 29
12.5 Reproducing the Simulink Results . . . 30

13 Software and Code **31**
13.1 Files . . . 31
13.2 Pseudo-Code . . . 31

13.3 Animations 34

14 Tabulation **34**

15 Conclusion **35**

16 Contributions **35**

1 Introduction

1.1 What is an Acrobot?

An acrobot is a two-link pendulum that hangs from a fixed pivot. The first link (the "shoulder" link) is free to rotate about the pivot; the second link is joined to the end of the first at the "elbow". The only motor sits at the elbow, so the shoulder is *passive*. It is the robot version of a gymnast on a high bar who can only swing the legs at the hips. Because there are two degrees of freedom but only one actuator, the acrobot is a standard example of an *underactuated* system [1], [2].

1.2 Objective

The project builds up, step by step, from a model to a robot that swings itself up and balances:

1. derive and verify the nonlinear model, find every equilibrium, and linearize about each one (Sections 2–4);
2. determine the stability of every equilibrium from the transfer functions and the state-space model, supported by simulation (Section 6);
3. decide whether feedback from a single measured joint angle can stabilize the upright, and explain why (Section 7);
4. design controllers for the upright and the other equilibria, and characterize how far from each equilibrium they still work with $\tau_{\max} = 20\,\mathrm{N\,m}$ (Sections 8–9);
5. starting from hanging, swing the acrobot up to the upright and hold it, and explain why the controller behaves as it does (Section 10);
6. compare the hand-designed swing-up with one learned by reinforcement learning (Section 11);
7. rebuild every model and controller (except the learned one) in Simulink and cross-check it against the Python results (Section 12).

2 System Modelling

2.1 Model

The configuration is described by two generalised coordinates: q_1, the angle of link 1 from the downward vertical, and q_2, the angle of link 2 relative to link 1. The full state contains the angles and their rates:

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}.$$

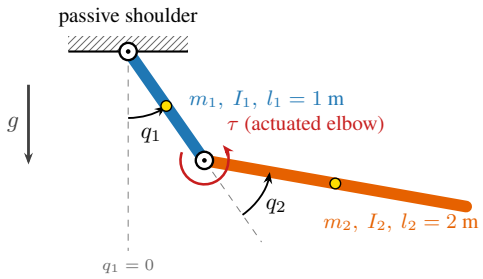


Figure 1: The acrobot: passive shoulder at the pivot, motor torque τ at the elbow; dots mark the centres of mass.

2.2 Equations of Motion

The dynamics (given in the course helper document and verified in Section 5) are

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} = \tau_g(q) + B\,\tau, \qquad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad |\tau| \le 20\,\mathrm{N\,m}. \tag{1}$$

τ **(Input Torque):** the torque of the elbow motor, the only quantity we command. $B = [0,\ 1]^T$ says it acts on the elbow equation only.

$M(q)$ **(Inertia Matrix):** the resistance to angular acceleration. It depends on the elbow angle because the mass distribution changes as the arm folds:

$$M(q) = \begin{bmatrix} 2.663 + 2\cos q_2 & 1.33 + \cos q_2 \\ 1.33 + \cos q_2 & 1.33 \end{bmatrix}, \qquad \det M = 1.7729 - \cos^2 q_2 \ge 0.7729 > 0. \tag{2}$$

$C(q,\dot{q})$ **(Coriolis Matrix):** the coupling forces between the links (centrifugal and Coriolis effects):

$$C(q,\dot{q}) = \sin q_2 \begin{bmatrix} -2\dot{q}_2 & -\dot{q}_2 \\ \dot{q}_1 & 0 \end{bmatrix}. \tag{3}$$

$\tau_g(q)$ **(Gravity Torque):** the torque that gravity exerts on each joint, $\tau_g = -\partial U/\partial q$, with potential energy U:

$$U = -14.715\cos q_1 - 9.81\cos(q_1 + q_2), \qquad \tau_g(q) = -9.81 \begin{bmatrix} 1.5\sin q_1 + \sin(q_1 + q_2) \\ \sin(q_1 + q_2) \end{bmatrix}. \tag{4}$$

System Parameters:

$$m_1 = m_2 = 1\,\mathrm{kg}, \qquad l_1 = 1\,\mathrm{m}, \qquad l_2 = 2\,\mathrm{m}, \qquad l_{c1} = 0.5\,\mathrm{m}, \qquad l_{c2} = 1\,\mathrm{m},$$
$$I_1 = 0.083\,\mathrm{kg\,m^2}, \qquad I_2 = 0.33\,\mathrm{kg\,m^2}, \qquad g = 9.81\,\mathrm{m\,s^{-2}}.$$

2.3 Derivation of the Model

The given model was checked from first principles with the Euler–Lagrange equations. Write $c_2 = \cos q_2$, $s_2 = \sin q_2$ and $\varphi = q_1 + q_2$.

1) Potential energy and gravity torque:

$$U = -(m_1 l_{c1} + m_2 l_1)\, g \cos q_1 - m_2 l_{c2}\, g \cos\varphi, \qquad \tau_g = -\frac{\partial U}{\partial q}. \tag{5}$$

2) Kinetic energy and inertia matrix: the velocity of the centre of link 2 is the elbow velocity ($l_1\dot{q}_1$) plus the rotation about the elbow ($l_{c2}\dot{\varphi}$), at angle q_2 apart (law of cosines):

$$T = \tfrac{1}{2}(m_1 l_{c1}^2 + I_1)\dot{q}_1^2 + \tfrac{1}{2} I_2 \dot{\varphi}^2 + \tfrac{1}{2} m_2 \Big(l_1^2 \dot{q}_1^2 + l_{c2}^2 \dot{\varphi}^2 + 2 l_1 l_{c2} c_2 \dot{q}_1 \dot{\varphi} \Big) = \tfrac{1}{2} \dot{q}^T M \dot{q}. \tag{6}$$

3) Coriolis term: with $\partial T/\partial \dot{q} = M\dot{q}$ and $\dot{M} = \frac{\partial M}{\partial q_2}\dot{q}_2 = -s_2 \dot{q}_2 \left[\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right]$,

$$\frac{d}{dt}\frac{\partial T}{\partial \dot{q}} - \frac{\partial T}{\partial q} = \tau_g + B\tau \;\Rightarrow\; M\ddot{q} + \underbrace{\Big(\dot{M}\dot{q} - \frac{\partial T}{\partial q} \Big)}_{= C\dot{q}} = \tau_g + B\tau. \tag{7}$$

These give exactly the M, C and τ_g of the given model.

2.4 State-Space Form

M is always invertible, so (1) can be solved for $\ddot{q}$:

$$\det M = 1.7729 - \cos^2 q_2 \geq 0.7729 > 0, \qquad \dot{x} = f(x,\tau) = \begin{bmatrix} \dot{q} \\ M^{-1}\big(\tau_g + B\tau - C\dot{q} \big) \end{bmatrix}. \tag{8}$$

3 Equilibria

3.1 Conditions

At an equilibrium the arm is at rest, $\dot{q} = \ddot{q} = 0$, and the motor torque is $\tau^* = 0$ (course specification). Then (1) reduces to $\tau_g(q^*) = 0$:

$$\text{row 2: } \sin(q_1^* + q_2^*) = 0, \qquad \text{row 1: } 1.5 \sin q_1^* + \underbrace{\sin(q_1^* + q_2^*)}_{=0} = 0 \;\Rightarrow\; \sin q_1^* = 0.$$

3.2 Solutions

$q_1^* \in \{0, \pi\}$ and $q_1^* + q_2^* \in \{0, \pi\}$, so $q_2^* \in \{0, \pi\}$ (mod 2π). This gives **four equilibria**, all with both links vertical:

Table 1: The four equilibria ($\tau^* = 0$)

	Posture	(q_1^*, q_2^*)	U [J]	Description
E_1	down–down	$(0, 0)$	-24.525	both links hanging
E_2	down–up	$(0, \pi)$	-4.905	link 1 down, link 2 up
E_3	up–up	$(\pi, 0)$	$+24.525$	fully upright (target)
E_4	up–down	(π, π)	$+4.905$	link 1 up, link 2 down

4 Linearization and Transfer Functions

4.1 Deviation Variables and Jacobians

With $\tilde{q} = q - q^*$, $\tilde{x} = x - x^*$ and $\tilde{\tau} = \tau$, a first-order Taylor expansion of (8) about $(x^*, 0)$ gives $\dot{\tilde{x}} \approx \frac{\partial f}{\partial x}\Big|_* \tilde{x} + \frac{\partial f}{\partial \tau}\Big|_* \tilde{\tau}$.

- **Top half:** $\frac{d}{dt}\tilde{q} = \dot{\tilde{q}}$, so the rows are $[\,0 \quad I\,]$.
- **Bottom half:** at $(x^*, 0)$ the Coriolis term drops out, $\partial(C\dot{q})/\partial\dot{q} = 0$, because $C\dot{q}$ is quadratic in $\dot{q}$. The derivative of M^{-1} multiplies $\tau_g + B\tau = 0$, so it drops out too. Only the gravity stiffness survives:

$$\ddot{\tilde{q}} = P\tilde{q} + r\tilde{\tau}, \qquad P = M(q^*)^{-1}K(q^*), \qquad r = M(q^*)^{-1}B, \tag{9}$$
$$K = \frac{\partial \tau_g}{\partial q}\Big|_{q^*} = -9.81 \begin{bmatrix} 1.5\cos q_1^* + \cos\varphi^* & \cos\varphi^* \\ \cos\varphi^* & \cos\varphi^* \end{bmatrix}.$$

4.2 Linear State-Space Model

$$\dot{\tilde{x}} = A\tilde{x} + B_\ell\tilde{\tau}, \quad y = C_y\tilde{x}, \qquad A = \begin{bmatrix} 0 & I \\ P & 0 \end{bmatrix}, \quad B_\ell = \begin{bmatrix} 0 \\ r \end{bmatrix}, \quad C_y = \begin{bmatrix} I & 0 \end{bmatrix}. \tag{10}$$

Example, E_1: $K = \left[\begin{smallmatrix} -24.525 & -9.81 \\ -9.81 & -9.81 \end{smallmatrix}\right]$, $M = \left[\begin{smallmatrix} 4.663 & 2.33 \\ 2.33 & 1.33 \end{smallmatrix}\right]$, $\det M = 0.7729$, so

$$P_{E1} = \begin{bmatrix} -12.6292 & 12.6926 \\ 14.7488 & -29.6119 \end{bmatrix}, \qquad r_{E1} = \begin{bmatrix} -3.0147 \\ 6.0332 \end{bmatrix}.$$

M depends only on q_2^* and K only on the signs of $\cos q_1^*$ and $\cos\varphi^*$. Upside down, gravity flips: $P_{E3} = -P_{E1}$ and $P_{E4} = -P_{E2}$.

Table 2: Linearization at the four equilibria ($\det M = 0.7729$ at all four)

	$(\cos q_1^*, \cos\varphi^*)$	$M(q^*)$	$P = M^{-1}K$	$r = M^{-1}B$
E_1	$(1, 1)$	$\left[\begin{smallmatrix} 4.663 & 2.33 \\ 2.33 & 1.33 \end{smallmatrix}\right]$	$\left[\begin{smallmatrix} -12.6292 & 12.6926 \\ 14.7488 & -29.6119 \end{smallmatrix}\right]$	$\left[\begin{smallmatrix} -3.0147 \\ 6.0332 \end{smallmatrix}\right]$
E_2	$(1, -1)$	$\left[\begin{smallmatrix} 0.663 & 0.33 \\ 0.33 & 1.33 \end{smallmatrix}\right]$	$\left[\begin{smallmatrix} -12.6292 & 12.6926 \\ 10.5095 & 4.2266 \end{smallmatrix}\right]$	$\left[\begin{smallmatrix} -0.4270 \\ 0.8578 \end{smallmatrix}\right]$
E_3	$(-1, -1)$	as E_1	$-P_{E1}$	r_{E1}
E_4	$(-1, 1)$	as E_2	$-P_{E2}$	r_{E2}

4.3 Transfer Functions

With zero initial conditions, $s^2\tilde{Q} = P\tilde{Q} + r\tilde{\mathcal{T}}$, so $(s^2 I - P)\tilde{Q}(s) = r\,\tilde{\mathcal{T}}(s)$ and

$$G_1(s) = \frac{r_1 s^2 + (P_{12}r_2 - P_{22}r_1)}{\Delta(s)}, \qquad G_2(s) = \frac{r_2 s^2 + (P_{21}r_1 - P_{11}r_2)}{\Delta(s)}, \tag{11}$$
$$\Delta(s) = s^4 - \operatorname{tr}(P)\, s^2 + \det P.$$

- **Poles:** the roots of Δ. With $\lambda = s^2$, $\lambda^2 - \mathrm{tr}(P)\lambda + \det P = 0$, then $s = \pm\sqrt{\lambda}$.
- **Zeros:** $s^2 = -(P_{12}r_2 - P_{22}r_1)/r_1$ for G_1 and $s^2 = -(P_{21}r_1 - P_{11}r_2)/r_2$ for G_2.
- With no friction only even powers of s occur, so poles and zeros come in pairs $\pm p$.

Table 3: Transfer functions at the four equilibria

	$\Delta(s)$	$N_1(s)$ $(\tau \to q_1)$	$N_2(s)$ $(\tau \to q_2)$	Zeros of G_1	Zeros of G_2
E_1	$s^4 + 42.2410\,s^2 + 186.7719$	$-3.0147\,(s^2 + 4.2103)$	$6.0332\,(s^2 + 5.2595)$	$\pm j2.0519$	$\pm j2.2934$
E_2	$s^4 + 8.4025\,s^2 - 186.7719$	$-0.4270\,(s^2 - 29.7273)$	$0.8578\,(s^2 + 7.3982)$	± 5.4523	$\pm j2.7200$
E_3	$s^4 - 42.2410\,s^2 + 186.7719$	$-3.0147\,(s^2 - 4.2103)$	$6.0332\,(s^2 - 5.2595)$	± 2.0519	± 2.2934
E_4	$s^4 - 8.4025\,s^2 - 186.7719$	$-0.4270\,(s^2 + 29.7273)$	$0.8578\,(s^2 - 7.3982)$	$\pm j5.4523$	± 2.7200

5 Simulation Environment and Model Verification

5.1 PyBullet Model

All closed-loop and open-loop results in this report come from the full nonlinear model in **PyBullet** [5]: uniform rods, no joint damping, step 0.25 ms, torque applied at the elbow only.

5.2 Verification

- **Model:** the derived M, C, τ_g match the given model term by term, and MATLAB reproduces every value in Tables 2 and 3 to four decimals.
- **PyBullet vs. the equations:** with $q(0) = (0.8, -0.5)$ rad and $\tau = 3\sin 2t$, PyBullet and an RK4 integration of (1) agree within 0.52° over 5 s. The error shrinks with the step size, so it is integration error, not a model mismatch.
- **Energy:** with $\tau = 0$, $T + U$ stays constant (no friction): to 10^{-8} with RK4 and to 1.2×10^{-3} J over 60 s in PyBullet.
- **Simulink:** an independent Simulink implementation reproduces the open-loop and closed-loop results to 10^{-10} degrees (Section 12).
- **Linear vs. nonlinear:** from the same small deviation the two agree within 1° (q_1) and 2° (q_2) over 8 s near E_1 (Fig. 2).

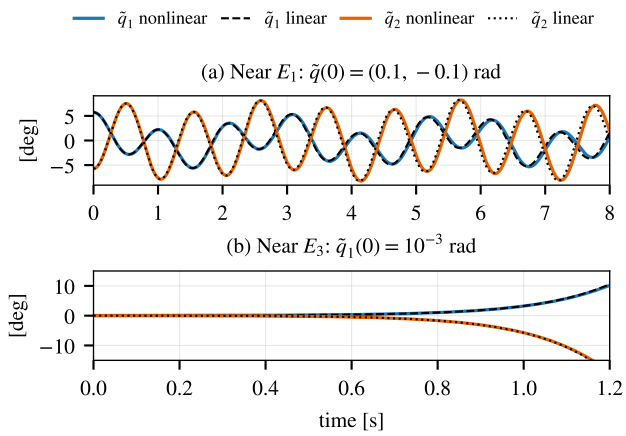


Figure 2: Nonlinear (colour) vs. linear (black) free responses near (a) E_1 and (b) E_3.

5.3 Interactive Simulator

The nonlinear model runs in PyBullet with an interactive dashboard (Fig. 3). The user picks an equilibrium, an initial condition (slider, typed value or by dragging the arm) and a controller: open loop, single-angle PD, single-angle observer + LQR, full-state LQR, pole placement, elbow PD, or swing-up → LQR. The dashboard shows the nonlinear arm next to the linear model, $\tilde{q}(t)$, $\tau(t)$ and the closed-loop poles in the s-plane, all updating live. Files: `phase2/acrobot_phase2_sim.py`, `phase3/acrobot_phase3_sim.py`, `phase4/acrobot_phase4_sim.py` (instructions in each folder's `README.md`).

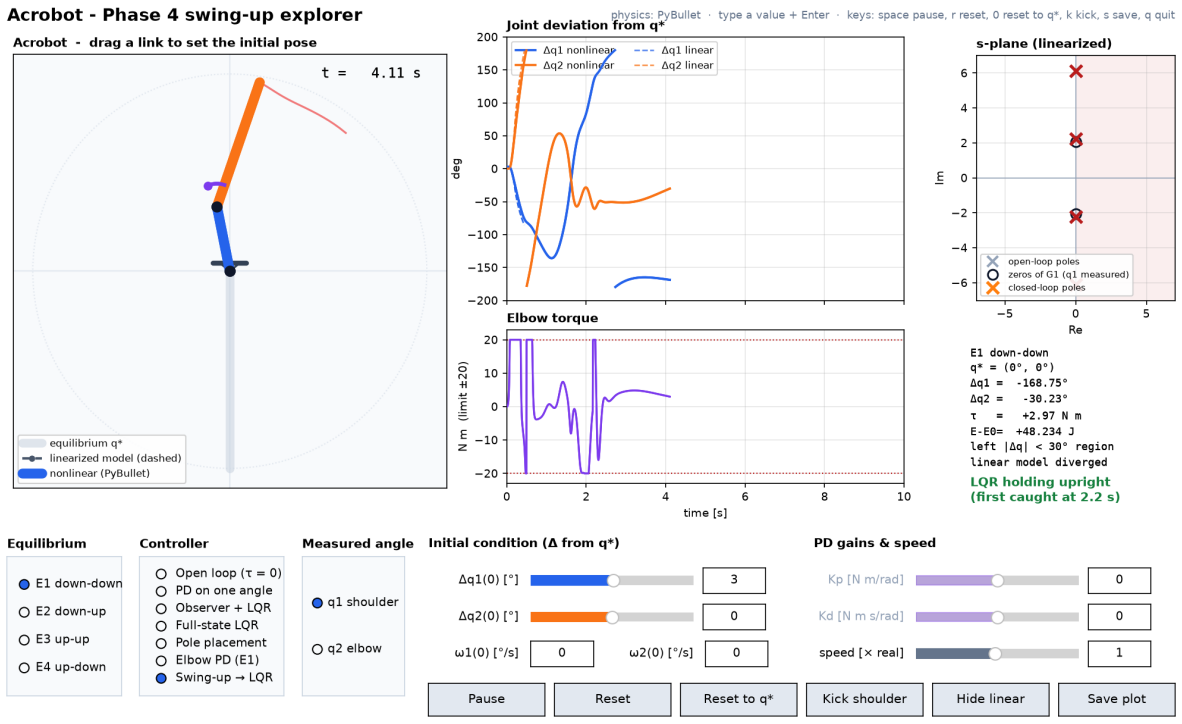


Figure 3: Interactive PyBullet dashboard during a swing-up from hanging.

6 Stability Analysis

6.1 Stability Criteria

- **State space:** the linear model (10) is *asymptotically stable* if every eigenvalue of A has $\operatorname{Re}\lambda < 0$, *unstable* if any has $\operatorname{Re}\lambda > 0$, and *marginally stable* if the eigenvalues on the imaginary axis are simple and none lies to the right.
- **Transfer function:** G_j is *BIBO stable* if every pole has $\operatorname{Re} p < 0$.
- **Nonlinear system (Lyapunov's indirect method):** an eigenvalue with $\operatorname{Re}\lambda > 0$ makes the equilibrium of (1) unstable; eigenvalues on the imaginary axis leave the question open [4].

6.2 Poles of the Transfer Functions = Eigenvalues of A

At every equilibrium

$$\operatorname{rank}[B_\ell \;\; AB_\ell \;\; A^2B_\ell \;\; A^3B_\ell] = 4, \qquad \operatorname{rank}\begin{bmatrix} C_j \\ C_jA \\ C_jA^2 \\ C_jA^3 \end{bmatrix} = 4 \quad (j = 1, 2),$$

so both G_1 and G_2 are minimal realizations with no pole–zero cancellation. Their poles are exactly the eigenvalues of A, and the transfer-function and state-space verdicts must coincide.

6.3 Characteristic Polynomial

$$\Delta(s) = s^4 + 0 \cdot s^3 - \operatorname{tr}(P)\, s^2 + 0 \cdot s + \det P. \tag{12}$$

A Hurwitz polynomial has all coefficients positive. Here the s^3 and s^1 coefficients are zero because the model has no friction, so **no equilibrium is asymptotically stable with** $\tau = 0$. Writing $\lambda = s^2$ (the eigenvalues of P):

$$\lambda^2 - \operatorname{tr}(P)\lambda + \det P = 0, \qquad \lambda < 0 \Rightarrow s = \pm j\sqrt{-\lambda}, \qquad \lambda > 0 \Rightarrow s = \pm\sqrt{\lambda}. \tag{13}$$

- $\det P > 0$, $\operatorname{tr} P < 0$: both $\lambda < 0$, all poles on the $j\omega$-axis (E_1).
- $\det P < 0$: one λ of each sign, one real RHP pole (E_2, E_4).
- $\det P > 0$, $\operatorname{tr} P > 0$: both $\lambda > 0$, two RHP poles (E_3).

6.4 Link to the Potential Energy

Since $K = -\nabla^2 U$ and $M \succ 0$, $P = M^{-1}K$ is similar to $M^{-1/2}KM^{-1/2}$, which is congruent to K. By Sylvester's law of inertia,

$$\#\{\text{RHP poles}\} = \#\{\text{negative eigenvalues of } \nabla^2 U\}. \tag{14}$$

A minimum of U gives no RHP pole, a saddle one, and a maximum two [3].

6.5 Stability of Each Equilibrium

Table 4: Stability of the four equilibria

	$(\operatorname{tr} P, \det P)$	poles of $G_1, G_2 = \operatorname{eig}(A)$	$\operatorname{eig}\nabla^2 U$	RHP	Verdict
E_1	$(-42.24,\ 186.77)$	$\pm j2.2400,\ \pm j6.1011$	$4.91,\ 29.43$ (min)	0	marginally stable
E_2	$(-8.40,\ -186.77)$	$\pm 3.1775,\ \pm j4.3010$	$-14.72,\ 9.81$ (saddle)	1	unstable
E_3	$(42.24,\ 186.77)$	$\pm 2.2400,\ \pm 6.1011$	$-29.43,\ -4.91$ (max)	2	unstable
E_4	$(8.40,\ -186.77)$	$\pm 4.3010,\ \pm j3.1775$	$-9.81,\ 14.72$ (saddle)	1	unstable

E_1 (down–down).

- **Transfer function:** the poles are simple and on the imaginary axis, so G_1 and G_2 are *not* BIBO stable. A bounded input at a resonance, $\tau = \sin(2.24t)$, gives a double pole at $\pm j2.24$ and a response that grows like $t\sin(2.24t)$.
- **State space:** the free response is a sum of two undamped modes ($\omega = 2.24$ and 6.10 rad/s): bounded but not decaying, so **marginally stable**.
- **Nonlinear:** linearization is inconclusive, so we use energy as a Lyapunov function:

$$V(x) = \tfrac{1}{2}\dot{q}^T M(q)\dot{q} + U(q) - U(q^*), \qquad \nabla^2 U(q^*) \succ 0 \Rightarrow V \succ 0, \qquad \dot{V} = \dot{q}^T B\tau = 0. \tag{15}$$

  E_1 is **stable in the sense of Lyapunov but not asymptotically stable**: the swing never decays.

E_2 and E_4 (one link up). $\det P = -186.77 < 0$, so each has one real RHP pole ($+3.1775$ at E_2, $+4.3010$ at E_4). Both are BIBO unstable, A is unstable, and by Lyapunov's indirect method the nonlinear equilibria are unstable.

E_3 (up–up). Gravity flips, $P_{E3} = -P_{E1}$, so λ changes sign and the poles become ± 2.2400, ± 6.1011: **two RHP poles**, unstable in every sense.

6.6 Simulation Evidence

All simulations use the full nonlinear model in **PyBullet** [5] (uniform rods, no damping, step 0.25 ms). Checked against an RK4 integration of (1) with $q(0) = (0.8, -0.5)$ rad and $\tau = 3\sin 2t$, the two agree to within 0.52° over 5 s.

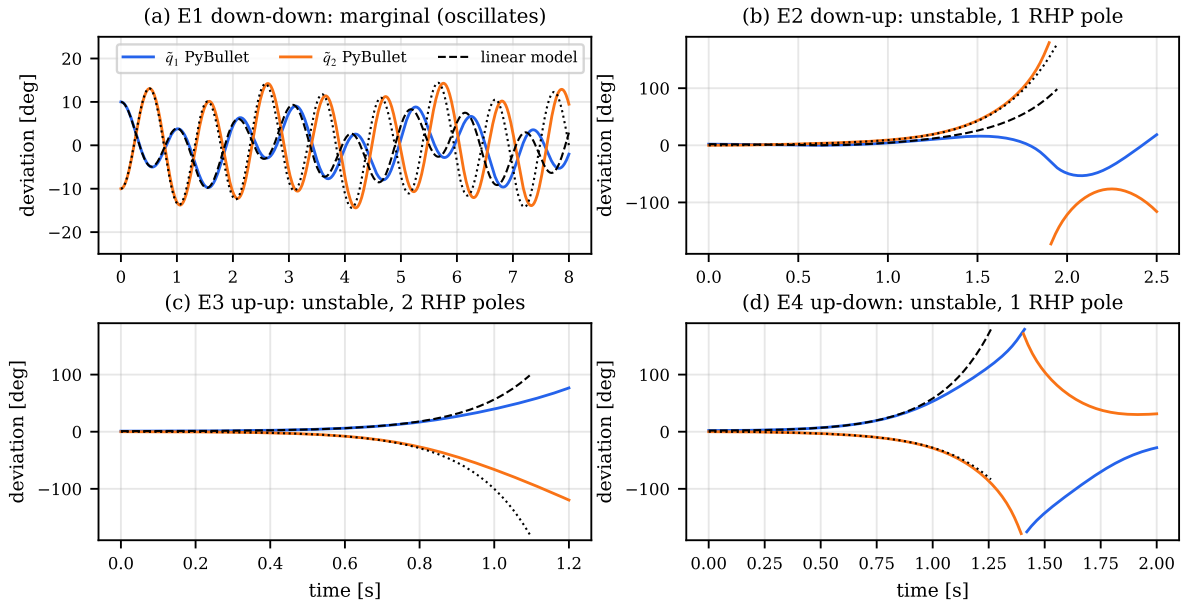


Figure 4: Free response ($\tau = 0$) near each equilibrium: PyBullet (colour) vs. linear model (black).

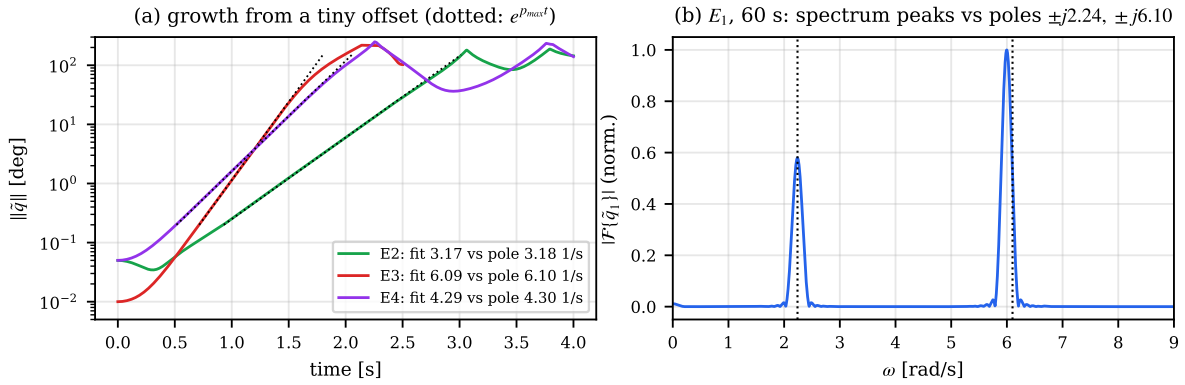


Figure 5: (a) Growth of a tiny offset on a log scale; dotted lines have the slope of the largest RHP pole. (b) Spectrum of $\tilde{q}_1$ near E_1; dotted lines are the poles.

Table 5: Linear predictions vs. PyBullet measurements

	Quantity	Predicted	Measured	Error
E_1	oscillation frequencies	2.240, 6.101 rad/s	2.239, 5.997 rad/s	within 1 bin
E_1	energy change over 60 s	0	1.2×10^{-3} J	integration error
E_2	growth rate	3.1775 s^{-1}	3.171 s^{-1}	0.22%
E_3	growth rate	6.1011 s^{-1}	6.088 s^{-1}	0.21%
E_4	growth rate	4.3010 s^{-1}	4.294 s^{-1}	0.15%

Observations:

- E_1 oscillates forever at its initial amplitude, with frequencies matching the poles.

- A tiny offset at E_2, E_3, E_4 grows exponentially at exactly the rate of the largest RHP pole. At E_3 a 0.01° tilt reaches 30° in 1.55 s.
- The linear model tracks the nonlinear one until about 20°, then over-predicts.

7 Feedback from a Single Measured Angle

7.1 Static or PD Feedback

Measuring one angle $y = \tilde{q}_j$ and applying $\tau = -(K_p + K_d s)\, y$ with $G_j = (b_2 s^2 + b_0)/(s^4 + a_2 s^2 + a_0)$ gives the closed-loop polynomial

$$s^4 + (K_d b_2)\, s^3 + (a_2 + K_p b_2)\, s^2 + (K_d b_0)\, s + (a_0 + K_p b_0). \tag{16}$$

Hurwitz requires $K_d b_2 > 0$ *and* $K_d b_0 > 0$, i.e. b_2 and b_0 of the same sign. At E_3:

- measuring q_1: $b_2 = -3.0147$, $b_0 = +12.6926$, opposite signs;
- measuring q_2: $b_2 = +6.0332$, $b_0 = -31.7316$, opposite signs.

The signs are opposite because the zeros ± 2.0519 and ± 2.2934 are **real**: one of each pair lies in the right half-plane. **No K_p, K_d can stabilize E_3 from one angle.** A grid search confirms it: the smallest attainable largest real part is 0.96 (q_1) and 2.29 (q_2), and in PyBullet even these gains fall from 1° within 2 s.

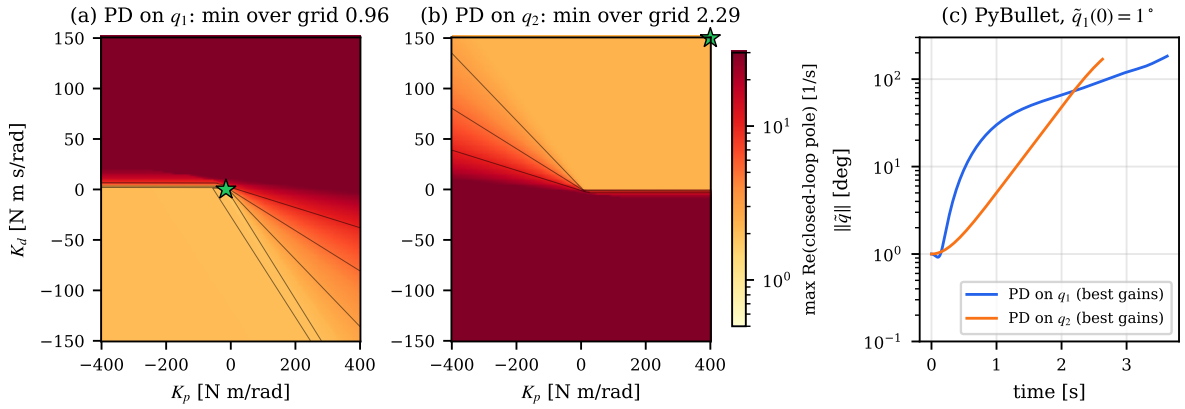


Figure 6: PD from one angle at E_3: (a, b) largest closed-loop real part over the gain plane, positive everywhere; (c) PyBullet with the best gains.

7.2 Dynamic (Observer-Based) Feedback

Each angle alone makes (A, C_j) observable, so an observer can reconstruct the full state and LQR can act on the estimate:

$$\dot{\hat{x}} = A\hat{x} + B_\ell \tau + L(y - C_j \hat{x}), \qquad \tau = -K\hat{x}, \qquad \lambda_{cl} = \lambda(A - B_\ell K) \cup \lambda(A - LC_j). \tag{17}$$

Two limits apply to *any* controller:

- **Parity interlacing [6]:** for q_1 the interval $(2.05, \infty)$ contains two real RHP poles, so a stable controller suffices. For q_2, $(2.29, \infty)$ contains one, so the controller itself must be unstable.
- **Sensitivity bound [3]:** $\|S\|_\infty \geq \prod_i |z + p_i|/|z - p_i|$, which is ≥ 45.9 for q_1 and ≥ 187 for q_2, because each zero sits just below the pole 2.24.

In PyBullet the observer-based loop holds E_3 only from 1.42° (measuring q_1) or 0.23° (measuring q_2), against 4.64° with full-state feedback.

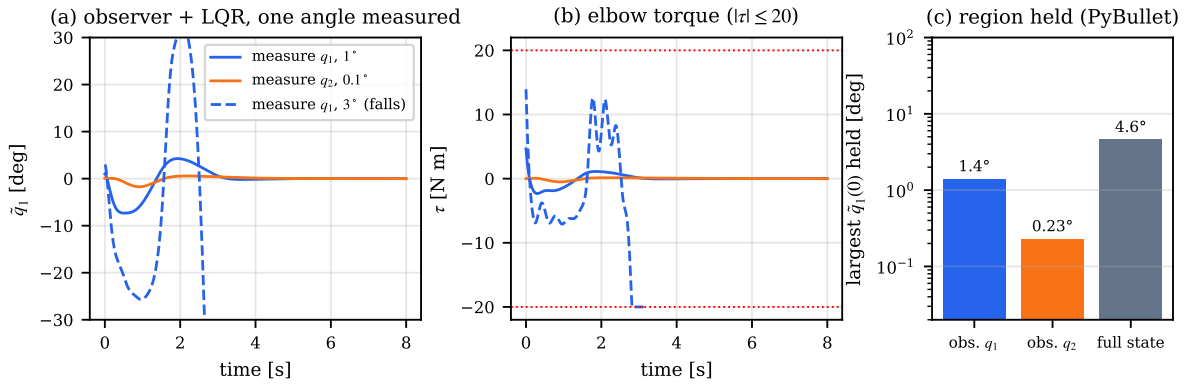


Figure 7: Observer + LQR at E_3 in PyBullet using one angle: (a) $\tilde{q}_1$; (b) torque; (c) largest initial offset held.

Answer: static or PD feedback from one joint angle **cannot** stabilize the upright, because of the real RHP zero. A dynamic controller **can**, but only for about a degree with $|\tau| \leq 20\,\mathrm{N\,m}$, so full-state feedback is used from here on (Section 8).

8 Controller Design Around the Equilibria

8.1 Controller Design

The rank of $\mathcal{C}$ is 4 at every equilibrium, so all four states can be fed back. Including the torque limit, every controller has the form

$$\tau = \mathrm{sat}(-K\tilde{x}), \qquad \tilde{x} = [\tilde{q}_1,\ \tilde{q}_2,\ \dot{q}_1,\ \dot{q}_2]^T, \qquad |\tau| \leq 20\,\mathrm{N\,m}. \tag{18}$$

The controllers used are:

- **LQR** at the upright E_3, at E_2 and at E_4;
- **pole placement** at E_3, for comparison;
- **PD on the elbow angle only** at E_1.

8.2 Linear Quadratic Regulator (LQR)

K minimizes a cost that penalizes both the state error and the torque:

$$J = \int_0^\infty \left(\tilde{x}^T Q \tilde{x} + R\tau^2\right) dt, \qquad A^T S + SA - SB_\ell R^{-1} B_\ell^T S + Q = 0, \qquad K = R^{-1} B_\ell^T S. \tag{19}$$

Expected performance of LQR:

- **Stability:** guaranteed for any $Q \succeq 0$, $R > 0$ when (A, B_ℓ) is controllable.
- **Robustness:** phase margin $\geq 60°$ and gain margin $[\frac{1}{2}, \infty)$ [7].
- **Tuning:** $Q = I$ weighs 1 rad of angle like 1 rad/s of rate. A smaller R makes the controller faster but demands more torque, so R trades speed against saturation.

8.2.1 Choice of R

For each R we simulated the nonlinear saturated loop from a grid of initial offsets and counted how many were recovered (Fig. 8).

- E_3**:** the recovered area stays at 16–20% of a $\pm 60°$ grid for $10^{-3} \leq R \leq 1$, while the fastest pole moves from -10 to -214 s^{-1}. The limit is $\tau_{\max}$, not the tuning. $R = 0.03$ is the knee: 93% of the best area with a fastest pole of -40.
- E_2**:** the region collapses for $R > 0.3$ (99% $\to$ 90% $\to$ 20%), so $R = 0.1$.
- E_4**:** the area peaks at $R \approx 0.1$–0.18, so $R = 0.1$.

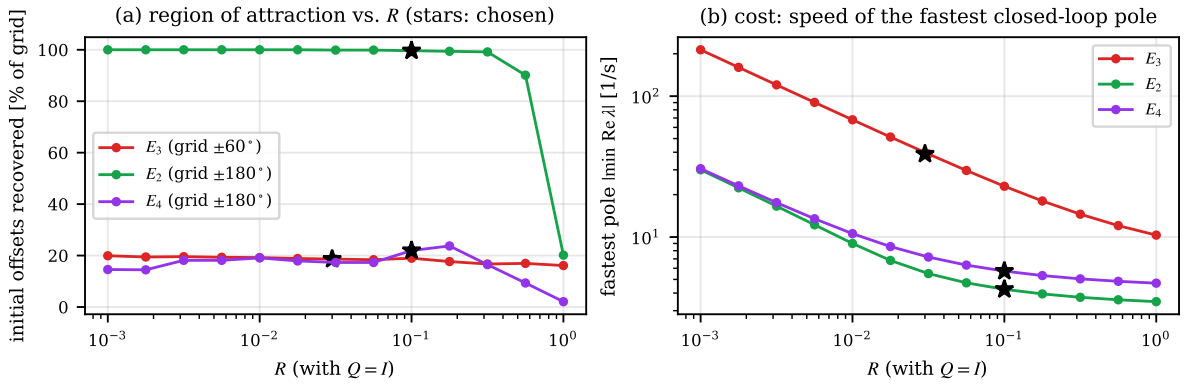


Figure 8: Weight selection ($Q = I$): (a) share of offsets recovered; (b) fastest closed-loop pole. Stars: chosen R.

8.3 Pole Placement at E_3 (Comparison)

Placing the poles at $\{-3, -4, -5, -6\}$ gives $K = [-322.9, -134.6, -138.1, -66.0]$. It settles faster from 2° (1.3 s vs 4.1 s) and with less torque (11.3 vs 17.6 N m), but recovers a smaller region (20% vs 26% of $\pm 40°$; 5° vs 6° along q_1). LQR is therefore kept for E_3.

8.4 E_1: PD on the Elbow Angle Only

At E_1 the zeros of G_2 are imaginary ($\pm j2.29$), so the sign conflict of (16) does not arise and one angle suffices:

$$\tau = -K_p\tilde{q}_2 - K_d\dot{q}_2, \qquad K_p = -5, \quad K_d = 0.4, \tag{20}$$

the pair with the best decay rate on a (K_p, K_d) grid ($\max \mathrm{Re}\,\lambda = -0.52$). **Proof of stability (energy shaping, LaSalle [4]):**

$$V_c = E - E^* + \tfrac{1}{2}K_p\tilde{q}_2^2, \qquad \dot{V}_c = -K_d\,\dot{q}_2^2 \le 0, \tag{21}$$
$$\nabla^2\Big(U + \tfrac{1}{2}K_p\tilde{q}_2^2\Big)\Big|_{E_1} \text{ has eigenvalues } 0.760,\ 28.575 > 0.$$

V_c has a strict minimum at E_1 and never increases, and $\dot{q}_2 \equiv 0$ only at E_1, so E_1 is asymptotically stable. Pure damping ($K_p = 0$) decays 19× more slowly.

8.5 Designed Controllers

Table 6: Designed controllers and their closed-loop poles

Controller	Design	$K = [k_{q_1}, k_{q_2}, k_{\dot{q}_1}, k_{\dot{q}_2}]$	Closed-loop poles
E_3 LQR	$Q = I$, $R = 0.03$	$[-503.0, -203.9, -218.3, -101.5]$	-39.86, $-2.25 \pm 0.10j$, -1.34
E_3 pole placement	$\{-3, -4, -5, -6\}$	$[-322.9, -134.6, -138.1, -66.0]$	-6, -5, -4, -3
E_2 LQR	$Q = I$, $R = 0.1$	[6.60, 46.49, 5.23, 13.24]	-4.26, -2.40, $-1.24 \pm 4.16j$
E_4 LQR	$Q = I$, $R = 0.1$	$[-49.31, 38.91, -13.19, 6.10]$	-5.73, -3.21, $-0.96 \pm 3.08j$
E_1 elbow PD	$K_p = -5$, $K_d = 0.4$	$[0, -5, 0, 0.4]$	$-0.69 \pm 2.31j$, $-0.52 \pm 2.14j$

9 Performance Limits under the Torque Limit

9.1 Guaranteed Region

Inside $\Omega_c = \{\tilde{x}^T S\tilde{x} \le c\}$ the command never saturates if

$$c^* = \frac{\tau_{\max}^2}{KS^{-1}K^T}, \qquad \dot{V} = -\tilde{x}^T(Q + K^T RK)\tilde{x} < 0 \ \text{ on } \Omega_{c^*}. \tag{22}$$

At rest this guarantees $0.72°/1.66°$ (E_3), $45.6°/19.9°$ (E_2) and $20.0°/23.1°$ (E_4) along q_1/q_2. The bound is conservative because short saturation is usually survived.

9.2 Simulated Region of Attraction

The real limits come from simulating the nonlinear saturated loop from a grid of initial offsets at rest. A start counts as recovered if it ends within 1° and 0.05 rad/s of q^*. The fast integrator used for the grids matches PyBullet on 100% of random points and 96% of boundary points.

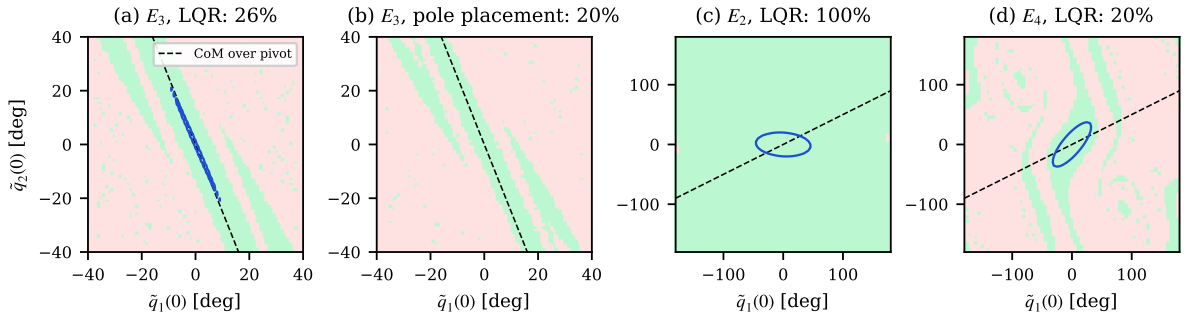


Figure 9: Region of attraction (green: recovered). Blue ellipse: guaranteed region. Dashed: offsets with the centre of mass over the pivot.

Table 7: How far from the equilibrium each controller still works

Controller	Recovered (grid)	$\lvert\tilde{q}_1(0)\rvert$ alone	$\lvert\tilde{q}_2(0)\rvert$ alone	Shoulder kick	Settling
E_3 LQR	26% ($\pm 40°$)	6°	13°	0.20 rad/s	4.1 s
E_3 pole placement	20% ($\pm 40°$)	5°	9°	0.15 rad/s	1.3 s
E_2 LQR	99.7% ($\pm 180°$)	175°	180°	≥ 8 rad/s	3.0 s
E_4 LQR	20.5% ($\pm 180°$)	35°	55°	2.1 rad/s	3.6 s
E_1 elbow PD	77.8% ($\pm 180°$)	40°	115°	2.15 rad/s	12.8 s

Why the upright region is a thin band. Linearizing τ_g at E_3:

$$\tau_{g,1} = 9.81\,(2.5\,\tilde{q}_1 + \tilde{q}_2). \tag{23}$$

Gravity pushes the passive shoulder in proportion to the horizontal offset of the total centre of mass. The motor can only counter this through the elbow coupling. Along $\tilde{q}_2 = -2.5\,\tilde{q}_1$ there is no shoulder torque, and the region reaches the grid edge ($\geq 43°$). Across that line, 20 N m is used up after about 6°.

9.3 Time Responses

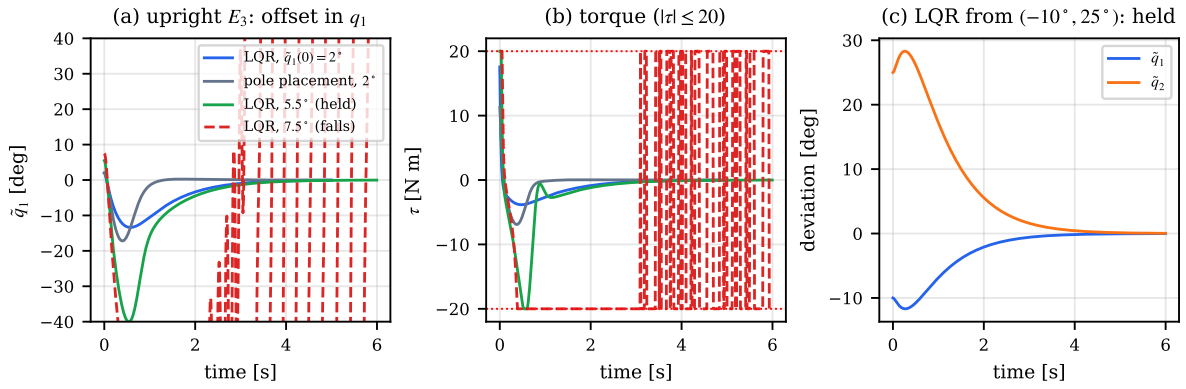


Figure 10: E_3 in PyBullet: LQR is held from 5.5° and falls from 7.5°; a CoM-balanced offset $(-10°, 25°)$ is held with only 3.3 N m.

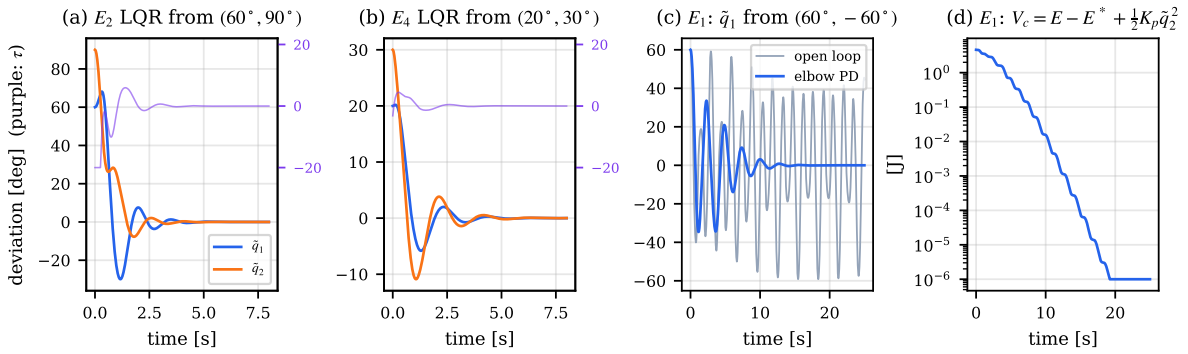


Figure 11: (a) E_2 and (b) E_4 with LQR; (c) E_1 with elbow PD vs. open loop; (d) V_c decreasing along the E_1 run.

Observations:

- **Settling time:** pole placement settles fastest at E_3, but LQR rescues a larger region. E_1 is slowest (12.8 s) because its slow mode barely moves the elbow.
- **Control effort:** at E_3 even a 2° offset needs 17.6 N m, close to the limit, which explains the small region. At E_4 the peak is only 4.5 N m.
- **Steady-state error:** zero for every controller (each converges to q^* within 0.001°).

10 Swing-Up Control

10.1 Energy Target

Hanging has $E = U(E_1) = -24.525$ J and the upright needs $E_{top} = U(E_3) = +24.525$ J. The controller must therefore add 49 J with a 20 N m elbow motor, and deliver the arm close enough to E_3 for the upright LQR of Section 8 to catch it.

10.2 Swing-Up Controller

Partial feedback linearization. With $h = C\dot{q} - \tau_g$, row 1 of (1) gives $\ddot{q}_1 = -(M_{12}\ddot{q}_2 + h_1)/M_{11}$. Substituting into row 2, the elbow follows any desired acceleration v exactly:

$$\tau = \left(M_{22} - \frac{M_{21}M_{12}}{M_{11}}\right)v + h_2 - \frac{M_{21}h_1}{M_{11}} \;\Rightarrow\; \ddot{q}_2 = v. \tag{24}$$

Energy pumping. The desired elbow acceleration is

$$v = -k_1\tilde{q}_2 - k_2\dot{q}_2 + k_3\,\dot{q}_1\,(E - E_{top}). \tag{25}$$

- k_1, k_2: keep the elbow nearly straight (a stiff and damped elbow).
- k_3: pushes in step with $\dot{q}_1$ while $E < E_{top}$ (adds energy) and against it when $E > E_{top}$ (removes energy), like a child pumping a swing.

10.3 Hybrid Switching

$$\tau = \text{sat}\begin{cases} -K\tilde{x}, & \tilde{x}^T S \tilde{x} < c_{sw} = 5 \quad \text{(catch: LQR at } E_3\text{, Sec. 8)} \\ \text{swing-up law above}, & \text{otherwise} \end{cases} \tag{26}$$

If the arm leaves the catch region (a missed catch or a push), swing-up resumes automatically.

10.4 Choice of Gains

We searched 144 designs ($k_1 \in \{20, 50, 100\}$, $k_2 \in \{2, 5, 10\}$, $k_3 \in \{0.3, 1, 3, 10\}$, $c_{sw} \in \{0.5, 1, 2, 5\}$) from 0.05 rad off hanging; 22 swung up and held. Each was then rerun from 40 random starts. $k = (20, 2, 3)$, $c_{sw} = 5$ succeeded from **39/40**, with a median first catch of 4.0 s.

10.5 Results

Table 8: Swing-up runs in PyBullet ($|\tau| \le 20\,\text{N m}$)

Start (from hanging)	First catch	Final deviation from E_3	Torque saturated	Result
$(3°, 0)$, $k = (20, 2, 3)$	2.2 s	$< 0.001°$	5%	held
$(-10°, 20°)$, $k = (20, 2, 3)$	9.5 s	$< 0.01°$	17%	held
$(3°, 0)$, kick 0.8 rad/s at 8 s	2.2 s, re-caught	$< 0.001°$	5%	held
$(3°, 0)$, textbook $k = (50, 5, 10)$	8.4 s	$< 0.001°$	7%	held (this start)

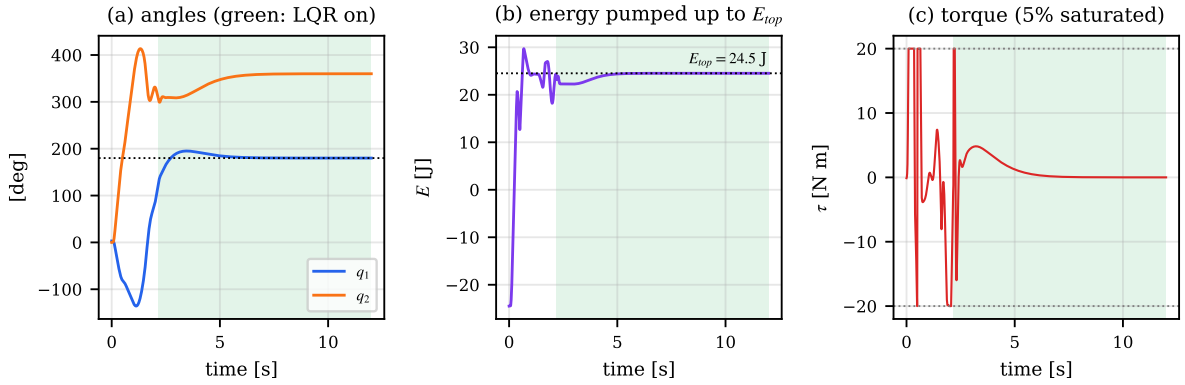


Figure 12: Swing-up from 3°: (a) angles, (b) energy pumped to E_{top}, (c) torque. Green: LQR active.

10.6 Why It Works, and Why Other Gains Fail

Table 9: Success from 40 random starts vs. saturation during swing-up

Gains (k_1, k_2, k_3), c_{sw}	Swing-up time at $\lvert\tau\rvert = 20$	Held (of 40)
$(20, 2, 3)$, 5 (chosen)	26%	**39**
$(50, 2, 3)$, 5	38%	4
$(100, 2, 3)$, 5	45%	2
$(20, 2, 0.3)$, 5	25%	0
$(50, 5, 10)$, 5 (textbook)	33%	25
$(50, 5, 10)$, 1 (textbook)	31%	13

1. **Saturation breaks the swing-up law.** The law assumes the full linearizing torque is applied. A stiff elbow (large k_1) asks for more than 20 N m more often (26% → 45% of the time). While saturated $\ddot{q}_2 \neq v$, the energy is no longer steered precisely, and the arm reaches the top off the catch region. Success drops from 39 to 2.

2. **Too little pumping.** With $k_3 = 0.3$ the energy approaches E_{top} too slowly to reach the catch region within 40 s (0/40).

3. **The catch is the LQR limit.** The LQR recovers only a thin region around E_3. The swing-up only has to deliver the arm into that region, and $c_{sw} = 5$ was the most reliable threshold tried.

4. **Retry.** Because swing-up resumes whenever $\tilde{x}^T S \tilde{x} \geq c_{sw}$, a missed catch or a push costs time, not the task (Fig. 14a).

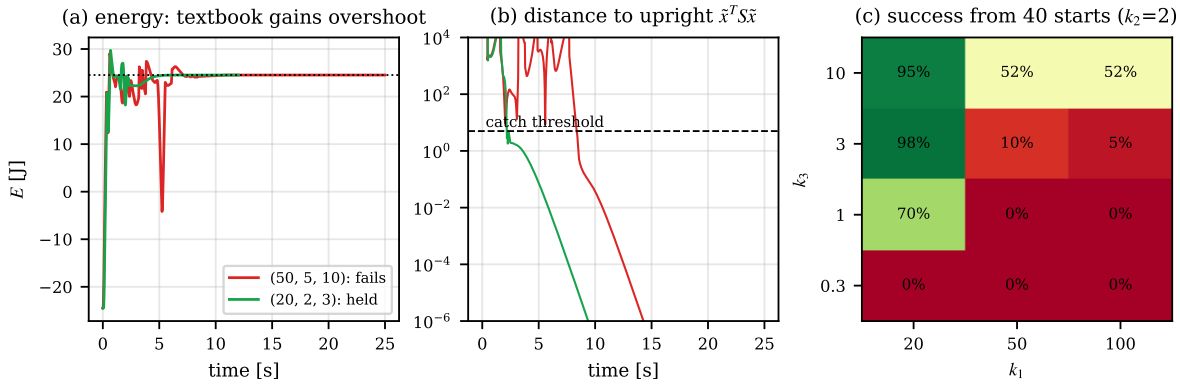


Figure 13: (a) Energy, chosen vs. textbook gains; (b) distance to the upright and the catch threshold; (c) success rate over (k_1, k_3).

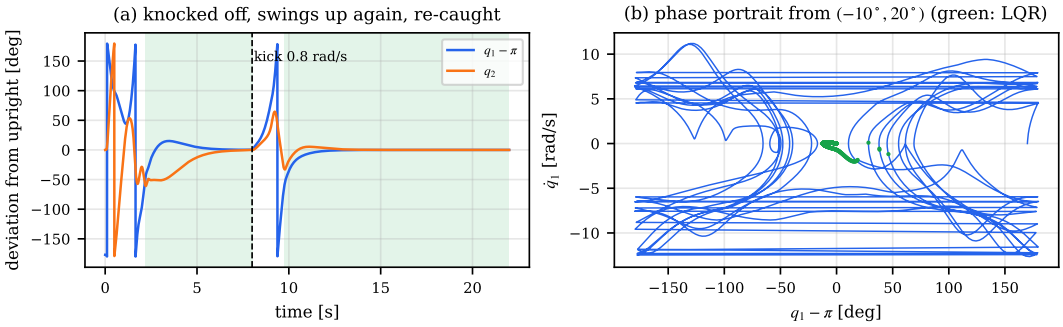


Figure 14: (a) Knocked off at 8 s, swung up and caught again; (b) phase portrait from $(-10°, 20°)$, green where LQR is active.

11 Learning-Based Swing-Up

11.1 Why Reinforcement Learning?

The energy swing-up of Section 10 works, but it is built entirely from the model: the partial feedback linearization needs $M(q)$, $C(q, \dot{q})$ and $\tau_g(q)$, and the energy target needs the exact masses and lengths. Reinforcement learning (RL) takes a different route. A neural-network policy learns *by trial and error in a simulator* which torque to apply, guided only by a score (the reward). No equation of motion is written into the controller.

- **Question:** can a learned swing-up match or beat the hand-designed one under the same 20 N m limit?
- **Fair comparison:** both controllers hand over to the *same* upright LQR of Section 8 inside the *same* catch zone, so the only difference is who does the swing-up.
- **Extra test:** what happens when the real robot differs from the model both controllers were designed or trained on?

11.2 Hybrid Structure

$$\tau = \begin{cases} \mathrm{sat}\left(-K\tilde{x}\right), & \tilde{x}^T S \tilde{x} < c_{sw} = 5 \quad \text{(catch: LQR, } R = 0.03\text{, Sec. 8)} \\ 20\,\pi_\theta(o), \quad \pi_\theta(o) \in [-1, 1], & \text{otherwise (learned swing-up, held for 20 ms)} \end{cases} \tag{27}$$

RL only has to deliver the arm into the catch zone; it never has to learn to balance. The part we can prove (the LQR near the top) stays model-based, and the hard nonlinear part is learned. If the arm leaves the zone, control returns to the policy, the same retry logic as the energy swing-up.

11.3 Environment

The environment follows the Gymnasium interface and uses the given equations of motion (1), integrated with RK4.

Table 10: RL environment

Element	Choice	Reason
Observation o	$[\cos q_1,\ \sin q_1,\ \cos q_2,\ \sin q_2,\ \dot{q}_1/10,\ \dot{q}_2/10]$	no jump at $\pm 180°$; inputs of order 1
Action a	$a \in [-1, 1]$, $\tau = 20\,a$	the torque limit is built in: the policy cannot exceed it
Control step	20 ms (50 Hz), four RK4 steps of 5 ms	realistic control rate
Start	$\lvert\tilde{q}_i(0)\rvert \le 0.6$ rad, $\lvert\dot{q}_i(0)\rvert \le 0.5$ rad/s around hanging	same spread as the robustness test of Section 10
Episode	15 s; ends early on entering the catch zone	short episodes, one clear goal

11.4 Reward Design

$$r = -\left(\frac{E - E_{top}}{E_{top}}\right)^2 - 0.3\,(1 + \cos q_1) - 0.1\,(1 - \cos q_2) \\ - 0.001\left(\frac{\tau}{20}\right)^2 + 100 \cdot \mathbb{1}[\text{enter catch zone}]. \quad (28)$$

- **Energy term:** the same target as the energy swing-up. It tells the policy *how much* energy to pump. Without it, RL tends to learn to spin the arm round and round, because a spinning arm passes near the top often.
- **Angle terms:** $1 + \cos q_1$ is 0 when link 1 points up; $1 - \cos q_2$ is 0 when the elbow is straight. They tell the policy *where* to go.
- **Torque term:** a small penalty that discourages wasted effort and bang–bang control.
- **Catch bonus:** +100 for reaching the region where the LQR takes over; the episode then ends.

11.5 Algorithm: Soft Actor-Critic (SAC)

SAC [8] learns a stochastic policy π_θ (the actor) and two action-value networks Q_ϕ (the critics) that maximize reward plus entropy:

$$J(\pi) = \mathbb{E}\Big[\sum_t \gamma^t \big(r_t + \alpha\,\mathcal{H}(\pi(\cdot \mid o_t))\big)\Big]. \quad (29)$$

The entropy term $\mathcal{H}$ keeps the policy exploring, and α is tuned automatically. SAC suits this problem because the torque is continuous and SAC reuses past experience (off-policy), so it needs relatively few simulated swings.

Table 11: Training settings (stable-baselines3 [9])

Setting	Value
Networks	2 hidden layers $\times$ 256 units (actor and critics)
Discount γ, learning rate, batch	0.99, 3×10^{-4}, 256
Replay buffer	300 000 transitions
Parallel environments, seed	4, 0
Training length	10^6 steps = 9 206 episodes, 12.4 min on a laptop CPU

11.6 Training

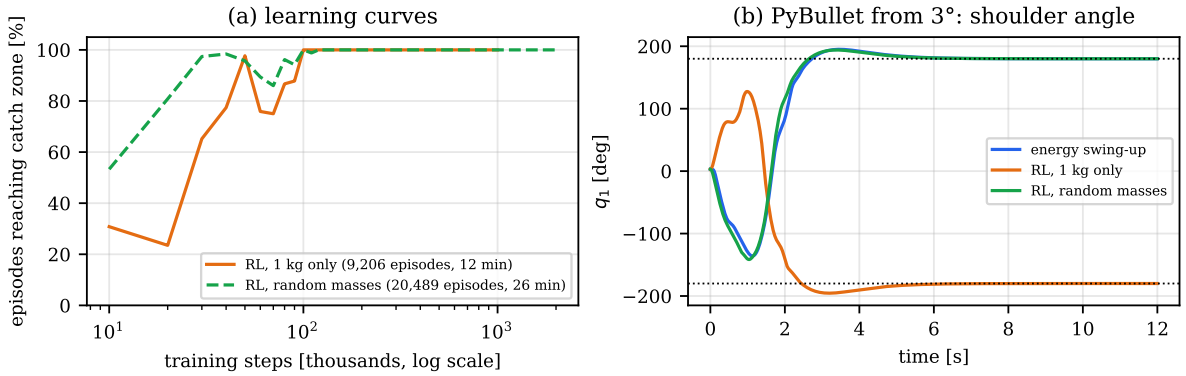


Figure 15: (a) Share of training episodes that reached the catch zone, for the original policy and for the robust policy of Section 11.9 (log scale). (b) PyBullet runs from 3°: the original RL policy swings the other way and ends at $q_1 = -180°$, the same upright pose as $+180°$; the robust policy swings the same way as the energy law.

Observations:

- The policy first reached the catch zone in every episode after about 10^5 steps (72 s of training) and stayed at 100% for the remaining 9×10^5 steps.
- Episodes kept getting shorter (more episodes per 10^4 steps: 54 at 10^5, 110 at 10^6): the policy kept learning to reach the top faster.
- The robust policy (Section 11.9), which faces a different robot in every episode, also first reached 100% after about 10^5 steps. Over its remaining 1.9×10^6 steps it never dropped below 98.8%.

11.7 Results: Correct Model

Both controllers were tested from the same 40 random starts as in Section 10. A start counts as held if it is caught and balanced within 30 s. (This is why the energy controller scores 35 rather than the value of 39 in Section 10, which allowed 40 s.)

Table 12: RL vs. energy swing-up, correct masses ($|\tau| \le 20\,\mathrm{N\,m}$)

Metric	Energy swing-up	RL swing-up (SAC)
Held (of 40)	35	**40**
Median time to first catch	3.98 s	**1.71 s**
Time at the torque limit	8.8%	**0.1%**
Median elbow work	244.9 J	**144.5 J**
PyBullet from 3°: first catch	2.17 s	**1.76 s**

Observations:

- **Reliability:** RL caught all 40 starts; the energy law caught 35 within 30 s.

- **Speed:** RL reaches the top more than twice as fast (median 1.71 s vs. 3.98 s).
- **Control effort:** RL almost never saturates the motor (0.1% vs. 8.8%) and uses 41% less work. Saturation was what broke the energy law (Section 10); the policy learned a timing that stays within 20 N m by itself.

11.8 Results: Wrong Model

The real robot never matches the model exactly. The same 40 starts were rerun with both link masses changed by ±20% (inertias scaled with them). Both controllers still assume 1 kg links.

Table 13: Robustness to the link masses (held of 40, within 30 s)

Link masses	Energy swing-up	RL swing-up (SAC)
−20% (0.8 kg)	37	38
correct (1.0 kg)	35	40
+20% (1.2 kg)	**35**	**15**

- **The energy law degrades gracefully** (35–37 of 40 in every case). It is built from physics: it pumps until $E = E_{top}$, so a heavier arm just takes a little longer (median 5.4 s instead of 4.0 s).
- **RL collapses on heavier links** (15 of 40). It practised only on 1 kg links, so it learned the *timing of that particular robot*, not the physics. It still reaches the catch zone in all 40 runs, but in a state from which the LQR cannot hold the heavier arm; it then repeats nearly the same approach (a median of 13 hand-overs in 30 s instead of 2).
- **The fix** is to let the policy practise on many different robots (Section 11.9).

11.9 Making the Policy Robust

A second policy was trained with exactly **two changes**. Everything else (network, reward, catch zone, LQR and SAC settings) is the same, so any difference in the results comes from these two changes.

Table 14: The two changes for the robust policy

	Original policy	**Robust policy**	**Why**
1. Link masses in training	always 1 kg	new random $m_1, m_2 \in [0.7, 1.3]$ kg in every episode	it cannot memorise one robot's timing
2. Observation	the 6 numbers of Table 10	the same 6, plus the missing energy $(E_{top} - E)/E_{top}$	the quantity the energy law steers by
Training length	10^6 steps, 9 206 episodes, 12.4 min	2×10^6 steps, 20 489 episodes, 25.9 min	many robots make a harder task

The extra input is computed from the other six with the nominal 1 kg model, exactly as the energy controller computes it. It therefore adds no new information: it is a hint that makes the important quantity explicit. Change 1 is the real fix (this is called *domain randomization*). By the end of training the robust policy reached the catch zone in 100% of episodes, as the original did.

All three swing-ups were then tested from the same 40 starts with five link masses. Two of them (0.6 and 1.4 kg) lie *outside* the range the robust policy practised on. Every controller still assumes 1 kg links.

Table 15: Held of 40 random starts (within 30 s) for five link masses

True link masses	Energy swing-up	RL, 1 kg only	RL, random masses
0.6 kg (−40%, never practised)	39	2	**40**
0.8 kg (−20%)	37	38	**40**
1.0 kg (correct)	35	**40**	**40**
1.2 kg (+20%)	35	15	**40**
1.4 kg (+40%, never practised)	**33**	3	2

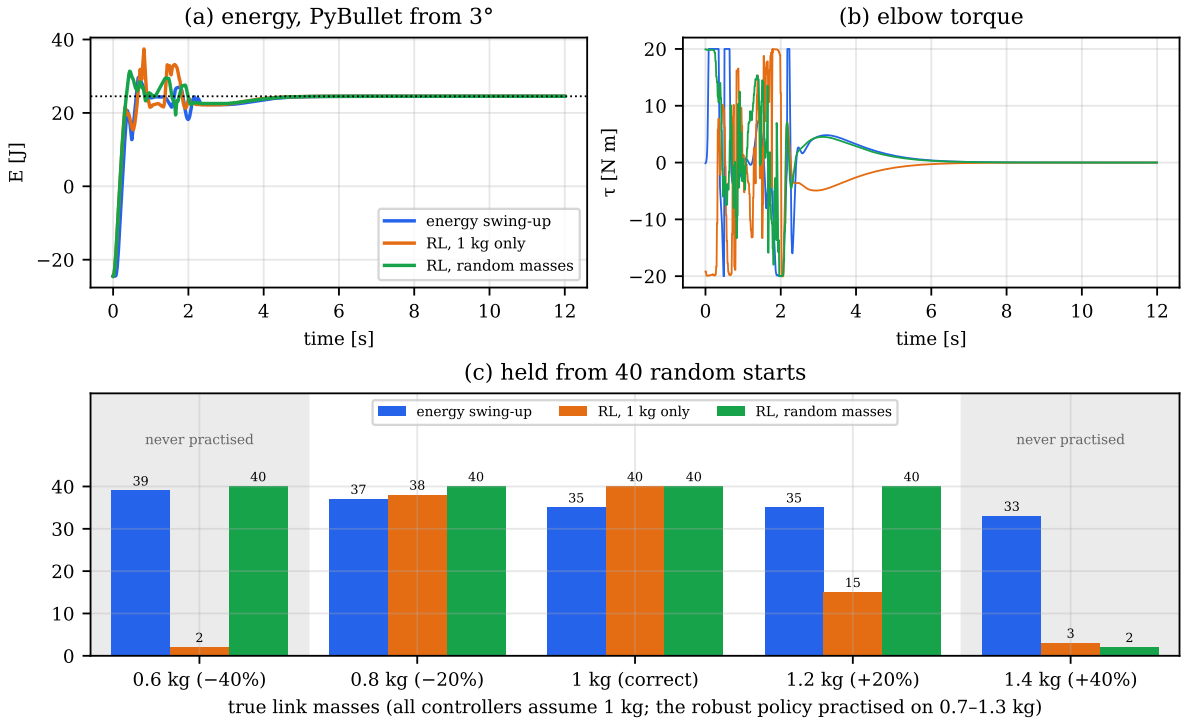


Figure 16: (a) Energy and (b) elbow torque in PyBullet from 3° for the three swing-ups; (c) starts held for five link masses. Shaded: masses the robust policy never practised on.

Observations:

- **Inside the practised range the problem is solved:** 40 of 40 at every mass from 0.8 to 1.2 kg. At +20% the score rises from 15 to 40.
- **No price on the correct model:** the robust policy also holds 40 of 40, with the same median catch time (1.71 s), no saturation (0.0%) and 10% less elbow work (130.0 J vs. 144.5 J). In PyBullet from 3° it catches at 1.89 s (original 1.76 s).

- **Lighter than ever practised (0.6 kg):** still 40 of 40. The original policy managed only 2, and in 25 of its 40 runs it never even reached the catch zone.
- **Much heavier than ever practised (1.4 kg):** only 2 of 40, while the energy law still holds 33. The policy reaches the catch zone in all 40 runs, but the hand-over fails again and again (a median of 12 hand-overs in 30 s, against 1 for the energy law). The energy law also loses many first catches, but each new approach is different, so a later catch usually works; the policy repeats nearly the same approach.
- **Why the hand-over is the weak point:** the reward ends the episode as soon as the arm *enters* the catch zone. The policy is never told whether the LQR then holds the arm, so it never learned to deliver the arm in an easy-to-catch state.

Lesson: a learned controller is reliable over the variety of robots it practised on (here, also somewhat beyond it on the light side). A physics-based law knows *why* it works, so it extrapolates further. Two natural next steps would be to reward holding rather than only arriving (end each episode only after the LQR has held the arm for, say, 1 s) and to widen the mass range.

11.10 Discussion

1. **Why RL is faster and gentler.** The energy law fixes the elbow motion through v and pays for it with saturation. The policy is free to choose any torque profile and is rewarded for reaching the top quickly with little effort, so it found a timing that pumps energy faster without hitting the limit.
2. **Why RL was fragile, and what fixed it.** Its knowledge is implicit in 256×256 hidden units fitted to the robots it practised on. Practising on one robot gave that robot's timing; practising on many made it robust over that range. Nothing in it says "pump until $E = E_{top}$", however, so far outside that range (1.4 kg) it can still fail where the energy law does not.
3. **Guarantees.** The LQR catch has a Lyapunov certificate (Section 9). The learned part has none: its performance is known only from tests, which is why it was tested on 40 starts, in a second simulator (PyBullet) and with five different link masses.
4. **Hybrid design.** Learning the hard nonlinear part and keeping the provable part near the top gave the best of both: the speed of RL, and a guaranteed final balance.

12 Simulink Implementation and Cross-Validation

Every result above was also rebuilt as a **Simulink** block diagram (MATLAB R2025b, Simulink 25.2). Simulink is the standard environment for control design in industry. A block diagram reads like the control loop on paper, the same model can later be turned into embedded code, and it gives a third, independent implementation of the plant and controllers next to the Python/PyBullet and MATLAB-script versions. Four models cover the open-loop, single-sensor, LQR and swing-up results; the learned (RL) controller is not included because it is a neural-network policy trained in Python.

12.1 Common Plant Structure

All four models share the same nonlinear plant loop. The torque command passes through a saturation block and enters a MATLAB Function block that evaluates the given equations of motion (1); an integrator closes the loop:

$$\tau_{\mathrm{cmd}} \xrightarrow{\mathrm{sat}_{\pm 20}} \tau \longrightarrow \dot{x} = f(x,\tau) \longrightarrow \int dt \longrightarrow x, \qquad x(0) = x_0. \tag{30}$$

Table 16: Blocks of the shared plant

Block	Simulink library block	Role
`tau limit`	Discontinuities / Saturation	actuator limit $\lvert\tau\rvert \le 20\,\mathrm{N\,m}$
`dynamics`	User-Defined Functions / MATLAB Function	$\ddot{q} = M(q)^{-1}\big(\tau_g(q) + B\tau - C(q,\dot{q})\dot{q}\big)$, returns $\dot{x}$
`x`	Continuous / Integrator ($1/s$)	state $x = (q_1, q_2, \dot{q}_1, \dot{q}_2)$, initial condition `x0`
`controller`	User-Defined Functions / MATLAB Function	control law (PD, LQR or hybrid swing-up)
Constants	Sources / Constant	gains and equilibrium (K, S, q^*, K_p, K_d, …) from the workspace
`x_out`, `tau_out`	Sinks / To Workspace	logged state and torque for analysis
`angles`	Sinks / Scope	live plot of q_1, q_2

The solver is the variable-step `ode45` (Dormand–Prince) with relative tolerance 10^{-7}, absolute tolerance 10^{-9} and a maximum step of 1 ms. Controller gains are not typed into the diagrams: they are computed in MATLAB (for example with `lqr`) and read from the workspace, so the same model runs at any equilibrium and with any gains.

12.2 Models

Table 17: Simulink models

Model	Topic	Controller block	Workspace inputs
`Acrobot_OpenLoop`	stability (Sec. 6)	none, $\tau = 0$; plus a linear State-Space block (A, B)	`A`, `B`, `x0`, `x0_lin`
`Acrobot_SingleAnglePD`	single sensor (Sec. 7)	$\tau = -(K_p e_j + K_d \dot{e}_j)$ on one angle j	`qs`, `Kp`, `Kd`, `j_meas`
`Acrobot_LQR`	control (Secs. 8–9)	$\tau = -K\tilde{x}$, angles wrapped to $(-\pi, \pi]$	`qs`, `K`
`Acrobot_SwingUp`	swing-up (Sec. 10)	energy pumping, LQR when $\tilde{x}^T S \tilde{x} < 5$	`K`, `S`, `gains`, `c_switch`

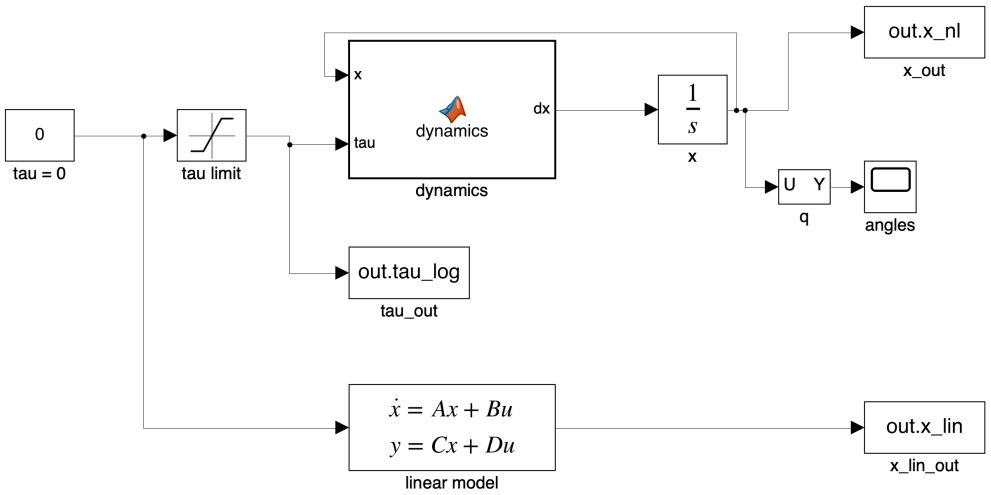


Figure 17: `Acrobot_OpenLoop`: the nonlinear plant (top) and the linearized State-Space model (bottom) driven by the same $\tau = 0$, so the two can be compared from the same deviation.

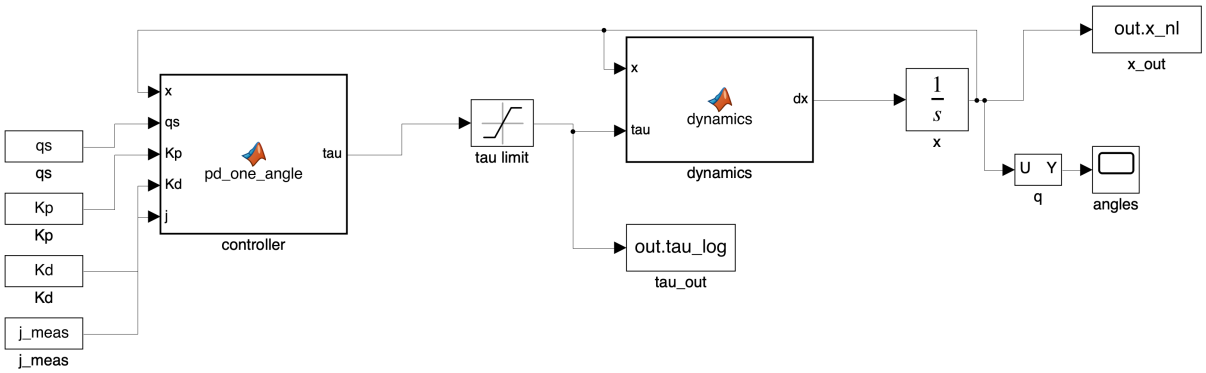


Figure 18: `Acrobot_SingleAnglePD`: PD feedback from one measured angle (`j_meas` = 1 or 2) at the upright.

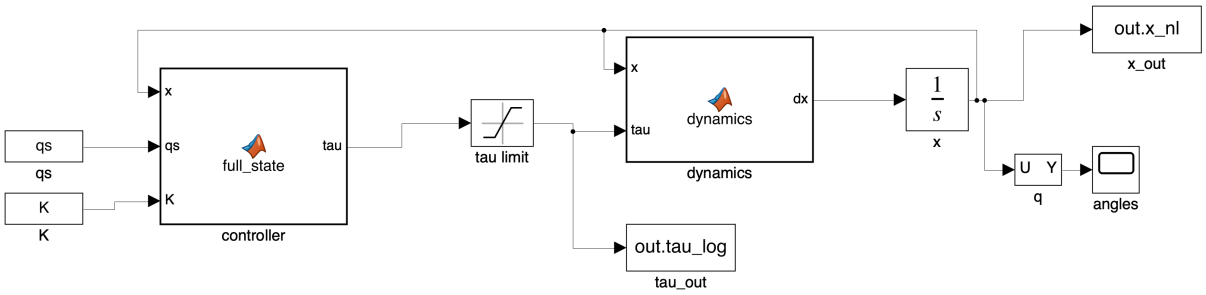


Figure 19: `Acrobot_LQR`: full-state feedback $\tau = \mathrm{sat}(-K\tilde{x})$ about the equilibrium `qs`.

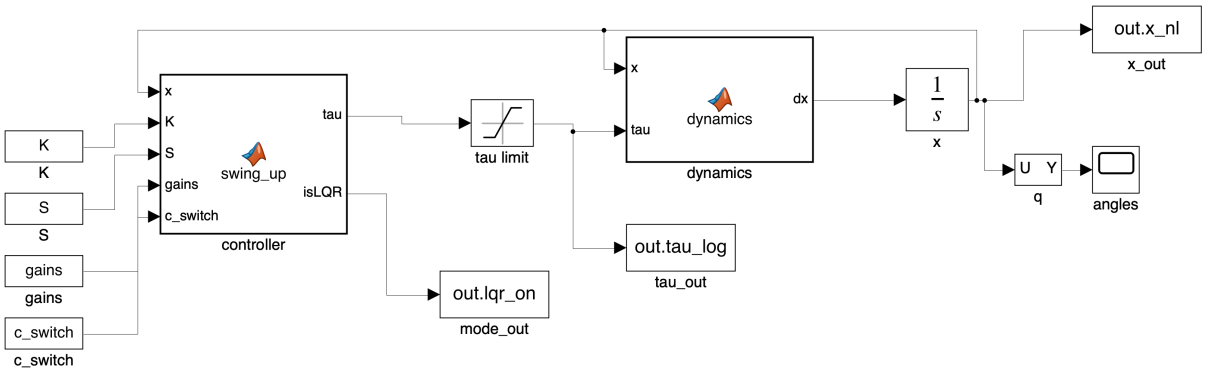


Figure 20: `Acrobot_SwingUp`: the hybrid controller has two outputs, the torque and the mode flag `isLQR` (logged as `lqr_on`), which becomes 1 once the state enters the catch region $\tilde{x}^T S \tilde{x} < 5$.

In the swing-up model the controller block holds both laws of Section 10. Outside the catch region it computes the energy error $E - E_{top}$ and the collocated partial-feedback-linearization torque; inside it applies the upright LQR. Because the switch is evaluated at every solver step, the solver handles the change of law as a discontinuity in τ, exactly like the real hybrid controller.

12.3 Results

The script `run_simulink_models.m` simulates every model with the gains and initial states used in this report and checks each outcome. All 15 checks pass (Table 18, Fig. 21).

Table 18: Simulink results (all checks pass)

Model	Case	Simulink result	Expected (this report)
OpenLoop	E_1, from $(10°, -10°)$	bounded, max. 14.7° over 8 s	marginally stable
OpenLoop	E_2	growth rate 3.171 /s	pole 3.1775 (error 0.19%)
OpenLoop	E_3	growth rate 6.088 /s	pole 6.1011 (error 0.22%)
OpenLoop	E_4	growth rate 4.300 /s	pole 4.3010 (error 0.03%)
SingleAnglePD	E_3, PD on q_1 ($K_p = -15$, $K_d = 0$)	falls	no single-angle PD works
SingleAnglePD	E_3, PD on q_2 ($K_p = 400$, $K_d = 150$)	falls	no single-angle PD works
LQR	E_3, $R = 0.03$, from 2°	held, peak $\lvert\tau\rvert = 17.56\,\mathrm{N\,m}$	held
LQR	E_3, from 5.5°	held, $\lvert\tau\rvert$ saturates	held
LQR	E_3, from 7.5°	falls	falls (beyond the 6° limit)
LQR	E_3, from $(-10°, 25°)$	held, peak $3.34\,\mathrm{N\,m}$	held (centre-of-mass line)
LQR	E_2, $R = 0.1$, from $(60°, 90°)$	held	held
LQR	E_4, $R = 0.1$, from $(20°, 30°)$	held, peak $4.47\,\mathrm{N\,m}$	held
SwingUp	from $(3°, 0°)$	caught at 2.17 s, held	caught at 2.17 s
SwingUp	from $(0.5°, 0°)$	caught at 1.90 s, held	held
SwingUp	from $(-10°, 20°)$	caught at 8.10 s, held	held

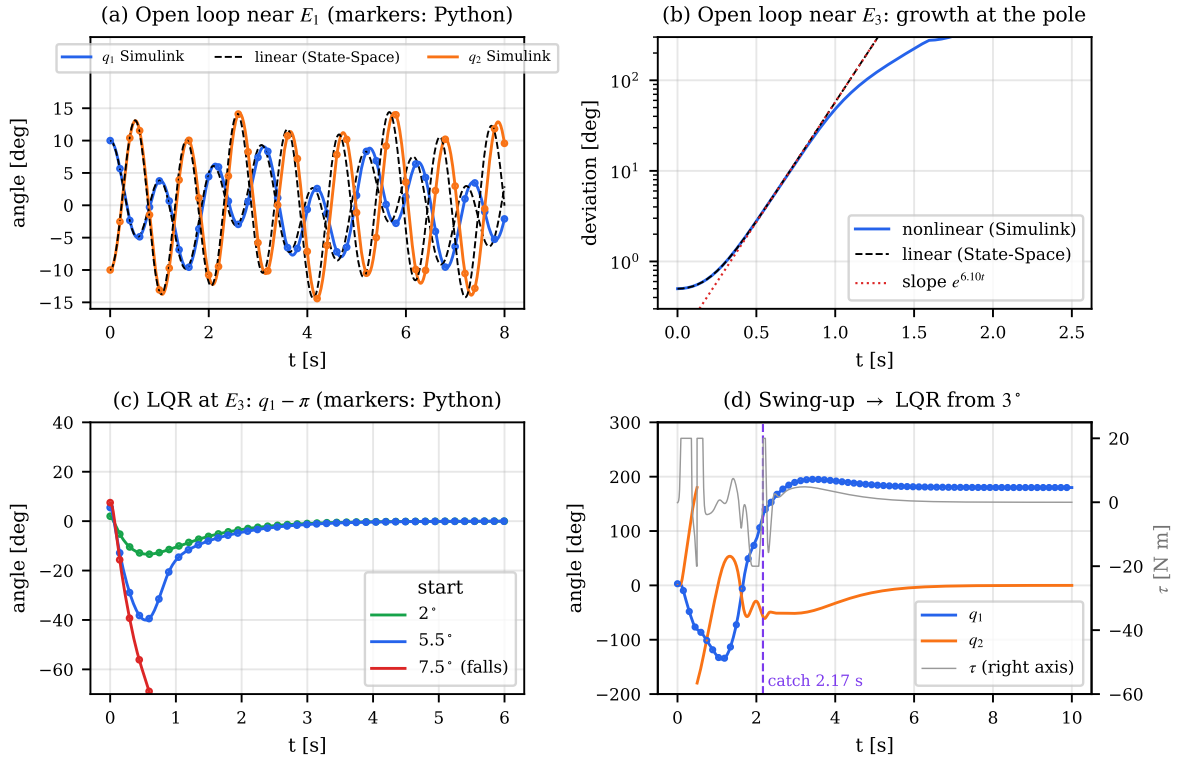


Figure 21: Simulink responses (lines) with the Python simulation of the same case (markers). (a) Free oscillation near E_1, nonlinear vs. linear State-Space block. (b) Near E_3 the deviation grows at the unstable pole 6.10/s until the nonlinearity takes over. (c) Upright LQR from 2°, 5.5° and 7.5°. (d) Swing-up from 3° and LQR catch at 2.17 s, with the saturated torque (grey, right axis).

12.4 Cross-Validation with the Python Model

Each Simulink trajectory was re-simulated from the same initial state with the Python model and controllers used for the rest of the report, integrated independently with fixed-step RK4 (steps of at most 0.1 ms, landing exactly on the Simulink time points). The

two implementations share only the equations, not the code or the solver.

Table 19: Simulink vs. Python (maximum angle difference over the whole run)

Case	Max. difference	Note
LQR gain K (MATLAB vs. Python `lqr`)	1.2×10^{-13} (relative)	identical design
Open loop near E_1, 8 s	6.9×10^{-11} deg	smooth
Open loop near E_3, 2.5 s	1.1×10^{-10} deg	smooth; falls from 0.5°
LQR at E_3 from 2°	4.2×10^{-11} deg	smooth
LQR at E_3 from 5.5°	6.0×10^{-11} deg	saturates briefly
LQR at E_3 from 7.5°	0.17 deg	both fall; long saturation
Swing-up from 3°, 10 s	0.0066 deg	catch 2.16691 s vs. 2.16693 s

Where the torque is smooth the two agree to 10^{-10} degrees, which is the level of the integration error of the two solvers. They differ more (but still well below a degree) only where the torque is discontinuous: at saturation and at the swing-up/LQR switch. There the variable-step solver and the fixed-step RK4 place the corner at slightly different instants, and the difference is then amplified by the unstable dynamics. Even so, both give the same outcome in every case, and the swing-up catch times agree to 2×10^{-5} s. The Simulink models therefore confirm the model, the stability results, the controllers and the swing-up independently of the Python and PyBullet code.

12.5 Reproducing the Simulink Results

The models are generated by a script rather than drawn by hand, so they can always be rebuilt identically:

- `build_simulink_models.m` creates the four `.slx` models and copies them into the code folders of the corresponding parts of the project;
- `run_simulink_models.m` runs the 15 checks of Table 18 and prints PASS/FAIL;
- `make_simulink_figures.m` exports the diagrams and the trajectories;
- `compare_with_python.py` re-simulates them in Python and produces Table 19 and Fig. 21.

Each model can also be opened and run by hand after setting its workspace inputs (Table 17).

13 Software and Code

13.1 Files

Table 20: Code files

Topic	File	Purpose
Model	`Acrobot_Nonlinear_Dynamics.m`	nonlinear model $\dot{x} = f(x, \tau)$ with the 20 N m limit
Model	`Acrobot_Linearize.m`	A, B_ℓ at an equilibrium
Stability	`Acrobot_Phase2_Main.m`	poles, zeros, ranks, verdicts, growth rates, single-angle PD test
Control	`Acrobot_Phase3_Design.m`, `Acrobot_Phase3_Main.m`	controllers, guaranteed regions, regions of attraction
Swing-up	`Acrobot_Phase4_Controller.m`, `Acrobot_Phase4_Main.m`	hybrid controller, swing-up and robustness tests
Learned swing-up	`acrobot_env.py`, `train.py`, `evaluate.py`	environment, SAC training, comparison (Python)
Simulator	`acrobot_phase{2,3,4}_sim.py`	interactive PyBullet dashboards
Simulink	`Acrobot_OpenLoop.slx`, `Acrobot_SingleAnglePD.slx`	open loop vs. linear model; single-angle PD
Simulink	`Acrobot_LQR.slx`, `Acrobot_SwingUp.slx`	full-state LQR; hybrid swing-up → LQR
Simulink	`build_simulink_models.m`, `run_simulink_models.m`	build the models; run the 15 checks
Simulink	`make_simulink_figures.m`, `compare_with_python.py`	diagrams, trajectories, Simulink vs. Python

13.2 Pseudo-Code

Algorithm 1 Nonlinear dynamics $\dot{x} = f(x, \tau)$ (`Acrobot_Nonlinear_Dynamics.m`) and linearization (`Acrobot_Linearize.m`)

function DYNAMICS($x = [q_1, q_2, \dot{q}_1, \dot{q}_2]$, τ)
  $\tau \leftarrow \max(-20, \min(20, \tau))$ ▷ motor limit
  build $M(q)$, $C(q, \dot{q})$ and $\tau_g(q)$ from the given model
  solve $M\,\ddot{q} = \tau_g + B\tau - C\dot{q}$ for $\ddot{q}$ ▷ backslash, never M^{-1}
  return $[\dot{q};\ \ddot{q}]$
end function
function LINEARIZE(q^*)
  $K \leftarrow \partial\tau_g/\partial q$ at q^*; $\quad P \leftarrow M(q^*)^{-1}K$; $\quad r \leftarrow M(q^*)^{-1}B$
  return $A = \begin{bmatrix} 0 & I \\ P & 0 \end{bmatrix}$, $B_\ell = \begin{bmatrix} 0 \\ r \end{bmatrix}$
end function

Algorithm 2 Stability analysis and single-angle test (`Acrobot_Phase2_Main.m`)

for each equilibrium $E_k \in \{E_1, E_2, E_3, E_4\}$ **do**
  $(A, B_\ell) \leftarrow$ LINEARIZE(q_k^*); poles $\leftarrow \mathrm{eig}(A)$
  $G_1, G_2 \leftarrow$ transfer functions $\tau \to q_1$, $\tau \to q_2$; zeros $\leftarrow$ roots of their numerators
  check $\operatorname{rank}\mathcal{C} = \operatorname{rank}\mathcal{O}_1 = \operatorname{rank}\mathcal{O}_2 = 4$ ▷ minimal: poles of $G_j = \mathrm{eig}(A)$
  verdict $\leftarrow$ asymptotically / marginally stable / unstable from the signs of Re(poles)
end for
for $E_k \in \{E_2, E_3, E_4\}$ **do** ▷ nonlinear check
  simulate $\tau = 0$ from a 0.01° offset (`ode45`)
  fit the slope of $\ln\|\tilde{q}(t)\|$ and compare it with the largest RHP pole
end for
for measured angle $j \in \{1, 2\}$ at E_3 **do** ▷ PD from one angle
  write $G_j = (b_2 s^2 + b_0)/\Delta(s)$
  if $b_2\, b_0 < 0$ **then**
    PD is impossible (no Hurwitz K_p, K_d)
  end if
  grid-search (K_p, K_d): record the smallest $\max \mathrm{Re}\,\lambda(A - B_\ell F)$
end for

Algorithm 3 Controller design and limits (`Acrobot_Phase3_Design.m`, `Acrobot_Phase3_Main.m`)

for each design (LQR at E_2, E_3, E_4; pole placement at E_3; elbow PD at E_1) **do**
  $(A, B_\ell) \leftarrow$ LINEARIZE(q^*)
  if LQR **then**
    $(K, S) \leftarrow$ `lqr`$(A, B_\ell, Q = I, R)$; $c^* \leftarrow \tau_{\max}^2/(KS^{-1}K^T)$ ▷ guaranteed region
  else if pole placement **then**
    $K \leftarrow$ `place`$(A, B_\ell, \{-3, -4, -5, -6\})$
  else
    $K \leftarrow [0,\ K_p,\ 0,\ K_d]$ ▷ elbow angle only
  end if
  print K and $\mathrm{eig}(A - B_\ell K)$
end for
function REGIONOFATTRACTION(K, q^*, grid of $(\tilde{q}_1(0), \tilde{q}_2(0))$)
  for each grid point **do**
    simulate $\tau = \mathrm{sat}(-K\tilde{x})$ for 10 s from rest ▷ nonlinear model
    mark it recovered if it ends within 1° and 0.05 rad/s of q^*
  end for
  return the map, the share recovered and the largest offset along each axis
end function

Algorithm 4 Hybrid swing-up controller (`Acrobot_Phase4_Controller.m`) and tests (`Acrobot_Phase4_Main.m`)

function CONTROLLER(x)
  $\tilde{x} \leftarrow$ deviation of x from the upright E_3 (angles wrapped to $\pm\pi$)
  if $\tilde{x}^T S \tilde{x} < c_{sw} = 5$ **then**
    $\tau \leftarrow -K\tilde{x}$ ▷ catch: upright LQR
  else ▷ swing-up: energy pumping
    $E \leftarrow \frac{1}{2}\dot{q}^T M \dot{q} + U(q)$
    $v \leftarrow -k_1\tilde{q}_2 - k_2\dot{q}_2 + k_3\,\dot{q}_1\,(E - E_{top})$
    $\tau \leftarrow (M_{22} - M_{21}M_{12}/M_{11})\,v + h_2 - M_{21}h_1/M_{11}$ ▷ partial feedback linearization
  end if
  return $\mathrm{sat}(\tau)$ ▷ $|\tau| \le 20\,\mathrm{N\,m}$
end function
for each start (three chosen starts, then 40 random starts near hanging) **do**
  simulate with fixed-step RK4 (1 ms) for 30–40 s, calling CONTROLLER every step
  record the first catch time, the share of time at $|\tau| = 20$, and whether E_3 is held
end for

Algorithm 5 Learned swing-up: environment step and SAC training (`acrobot_env.py`, `train.py`)

function RESET
  $x \leftarrow$ random start near hanging; $m_1, m_2 \leftarrow 1$ kg ▷ robust policy: $m_i \sim U[0.7, 1.3]$ kg
end function
function ENVSTEP(x, action $a \in [-1, 1]$)
  $\tau \leftarrow 20a$; integrate the model with masses m_1, m_2 for 20 ms (four RK4 steps of 5 ms)
  $r \leftarrow -((E - E_{top})/E_{top})^2 - 0.3(1 + \cos q_1) - 0.1(1 - \cos q_2) - 0.001(\tau/20)^2$
  if $\tilde{x}^T S \tilde{x} < 5$ **then**
    $r \leftarrow r + 100$; episode ends ▷ reached the LQR catch zone
  end if
  $o \leftarrow [\cos q_1, \sin q_1, \cos q_2, \sin q_2, \dot{q}_1/10, \dot{q}_2/10]$ ▷ robust policy: append $(E_{top} - E)/E_{top}$
  return o, r
end function
initialise actor π_θ, critics Q_{ϕ_1}, Q_{ϕ_2} (2×256 each), empty replay buffer
for 10^6 environment steps (4 environments in parallel) **do**
  $a \sim \pi_\theta(\cdot \mid o)$; $(o', r) \leftarrow$ ENVSTEP(x, a); store (o, a, r, o') in the buffer
  sample a batch of 256; update the critics towards $r + \gamma(\min_i Q_{\phi_i}(o', a') - \alpha \log \pi_\theta(a' \mid o'))$
  update the actor to maximise $\min_i Q_{\phi_i}(o, a) - \alpha \log \pi_\theta(a \mid o)$; tune α
end for
save π_θ

Algorithm 6 Learned swing-up: RL → LQR controller and evaluation (`acrobot_env.py`, `evaluate.py`)

function RLController(x)
  if $\tilde{x}^T S \tilde{x} < 5$ **then**
    return sat$(-K\tilde{x})$ ▷ same upright LQR catch
  end if
  every 20 ms: $\tau \leftarrow 20\,\pi_\theta(o)$ (deterministic); hold it in between
  return τ
end function
for link masses $\in \{0.6,\ 0.8,\ 1.0,\ 1.2,\ 1.4\}$ kg **do** ▷ controllers always assume 1 kg
  for each of the 40 random starts, for the energy and both RL controllers **do**
    simulate 30 s; record held, first catch time, torque saturation, elbow work and hand-overs
  end for
end for

13.3 Animations

2K Manim animations of the work: model and equilibria, stability, control, swing-up; the comparison of the learned and energy swing-ups is `phase5_rl/RL_vs_Energy_SwingUp_2K.mp4`.

14 Tabulation

Table 21: Summary of the stability analysis

Equilibrium	RHP poles	Energy shape	Stability
E_1 down–down	0	minimum	marginally (Lyapunov) stable
E_2 down–up	1	saddle	unstable
E_3 up–up	2	maximum	unstable
E_4 up–down	1	saddle	unstable

Table 22: Summary of every controller ($|\tau| \le 20\,\mathrm{N\,m}$)

Controller	Works?	Limit
PD on one angle at E_3	no (proved)	always unstable
Observer + LQR, one angle, at E_3	yes	1.42° (q_1), 0.23° (q_2)
Full-state LQR at E_3	yes	6° / 13°, ≥ 43° along the CoM line
Pole placement at E_3	yes	5° / 9°
LQR at E_2	yes	almost any rest position (99.7%)
LQR at E_4	yes	35° / 55°
Elbow PD at E_1	yes	78% of ±180°
Energy swing-up → LQR	yes	39/40 random starts (40 s), 35/40 (30 s)
RL (SAC) swing-up → LQR, 1 kg only	yes	40/40 (30 s); 15/40 with links +20% heavier
RL swing-up → LQR, random masses	yes	40/40 from 0.6 to 1.2 kg; 2/40 at 1.4 kg

15 Conclusion

The given acrobot model was verified from first principles. It has four zero-torque equilibria, all with both links vertical. Only the hanging pose is stable, and only in the Lyapunov sense: with no friction it oscillates forever. The other three are unstable, with one, two and one right-half-plane poles, matching the number of directions in which the potential energy decreases. The transfer functions and the state matrix agree because every realization is minimal, and PyBullet confirmed the predicted growth rates to within 0.25%.

A single measured angle cannot stabilize the upright with PD feedback, because the transfer functions have real right-half-plane zeros. An observer-based controller can, but only for about a degree. With full-state feedback all four equilibria were stabilized. At the upright the 20 N m limit, not the tuning, restricts LQR to a thin band around the line where the centre of mass is over the pivot.

An energy-pumping swing-up with partial feedback linearization brings the arm from hanging to the upright, and the LQR catches and holds it from 39 of 40 random starts. Stiffer gains fail because saturation breaks the linearization. A swing-up learned with reinforcement learning was faster, gentler and more reliable on the model it was trained on, but much less robust when the link masses changed. Training it on random link masses, with the missing energy as one extra input, fixed this over the practised range (40 of 40 from 0.6 to 1.2 kg) at no cost on the correct model; far outside that range (1.4 kg) the physics-based law remained the more reliable one. The hybrid design (learn or design the swing-up, keep the provable LQR catch) proved the most useful structure throughout.

Finally, every model and controller except the learned one was rebuilt in Simulink. The Simulink models pass all 15 checks and agree with the independent Python simulation to 10^{-10} degrees wherever the torque is smooth, and to well below a degree across saturation and the swing-up switch, so the results do not depend on one particular piece of code.

16 Contributions

1. Report compilation – _____
2. Modelling, equilibria and linearization (Sections 2–4) – _____
3. Simulation environment and verification (Section 5) – _____
4. Stability analysis and single-sensor feedback (Sections 6–7) – _____
5. Controller design and limits (Sections 8–9) – _____
6. Swing-up control (Section 10) – _____
7. Learning-based swing-up (Section 11) – _____
8. Simulink models and cross-validation (Section 12) – _____
9. Code, simulator and animations (Section 13) – _____

References

[1] M. W. Spong, "The swing up control problem for the Acrobot," *IEEE Control Systems Magazine*, vol. 15, no. 1, pp. 49–55, 1995.

[2] R. Tedrake, *Underactuated Robotics*, MIT, 2024. Available: https://underactuated.csail.mit.edu

[3] S. Skogestad and I. Postlethwaite, *Multivariable Feedback Control*, 2nd ed., Wiley, 2005.

[4] H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, 2002.

[5] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation," 2016–2024. Available: https://pybullet.org

[6] D. C. Youla, J. J. Bongiorno and C. N. Lu, "Single-loop feedback-stabilization of linear multivariable dynamical plants," *Automatica*, vol. 10, pp. 159–173, 1974.

[7] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*, Prentice Hall, 1990.

[8] T. Haarnoja, A. Zhou, P. Abbeel and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018.

[9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *JMLR*, vol. 22, no. 268, pp. 1–8, 2021.